

SASD Math Toolkit

Numerical Methods - C# / .NET

Benutzer- und Entwicklerhandbuch - Version 1.0 RC1

Numerische Verfahren für robuste, nachvollziehbare und wiederverwendbare C#-Anwendungen.

Repository: github.com/Robin-Goerlach/SASD-Math-Toolkit | C#/.NET

Dokumentstatus

Handbuch Version 1.0 RC1

Dieses Dokument ist die deutschsprachige Release-Candidate-Ausgabe des SASD Math Toolkit zum Paketkandidaten 1.0.0-rc.1. Die Kapitelinhalte werden beim Build aus dem gepflegten Markdown-Handbuch unter docs/user-guide/de/ erzeugt. Markdown bleibt damit die redaktionelle Quelle, während LaTeX Satz, Navigation und PDF-Ausgabe übernimmt.

Dokument	SASD Math Toolkit - C#/.NET Benutzer- und Entwicklerhandbuch
Handbuchversion	1.0 RC1
Paketkandidat	1.0.0-rc.1
Repository	github.com/Robin-Goerlach/SASD-Math-Toolkit
Zielpattform	.NET 10
Assembly	Sasd.Math.Toolkit
Root Namespace	Sasd.Numerics
Lizenz	MIT
Autor / Projekt	Robin Goerlach; SASD-GmbH

Vorwort

Das SASD Math Toolkit überträgt den funktionalen Kern einer klassischen Numerical-Methods-Toolbox in eine moderne, unabhängig entwickelte C#/.NET-Bibliothek und erweitert diese Basis anschließend gezielt für Anforderungen des Jahres 2026. Das historische Handbuch dient als Orientierung für Problemklassen und Handbuchdramaturgie - nicht als Text- oder Quellcodevorlage.

Die Kapitel 1-11 beschreiben die klassische numerische Grundlage. Die Kapitel 12 und 13 dokumentieren die moderne dichte und dünnbesetzte lineare Algebra der 2026-Modernisierung: Householder-QR, Cholesky, SVD, Konditionsdiagnostik, kanonisches CSR, Preconditioning, CG/PCG, restarted GMRES und BiCGSTAB.

Leitgedanke

Eine Zahl ist noch kein belastbares numerisches Ergebnis. Für wichtige Rechnungen gehören Methode, Toleranz, Terminierungsstatus und eine problemspezifische Qualitätsprüfung zusammen.

Inhaltsverzeichnis

Dokumentstatus	iii
Vorwort	v
1 Erste Schritte mit C#/.NET	1
1.1 Aktuelle Entwicklungsform	1
1.2 Namespaces	1
1.3 Eine erste Berechnung	1
1.4 Grafische Numerik-Demo starten	1
1.5 Fehlerbehandlung	2
2 Nullstellen von Gleichungen	3
2.1 Welches Verfahren passt zu welchem Problem?	3
2.2 Gemeinsame Optionen und Ergebnisdaten	3
2.3 Bisektion: das robuste Intervallverfahren	4
2.4 Newton-Raphson: schnell bei gutem lokalem Modell	5
2.5 Sekantenverfahren: lokale Iteration ohne Ableitungsfunktion	5
2.6 Polynome und Horner-Schema	5
2.7 Muller: von reellen Startwerten in die komplexe Ebene	6
2.8 Laguerre für Polynomnullstellen	6
2.9 Konvergenz richtig interpretieren	7
2.10 Sinnvoller Arbeitsablauf	7
3 Interpolation	9
3.1 Welches Verfahren wählen?	9
3.2 Lagrange-Interpolation	9
3.3 Newton und dividierte Differenzen	10
3.4 Natürlicher kubischer Spline	10
3.5 Geklemmter kubischer Spline	11
3.6 Definitionsintervall und Unveränderlichkeit	11
3.7 Ungültige Daten und numerische Probleme	11
3.8 Interpolation ist keine Approximation	12
4 Numerische Differentiation	13

4.1	Eine Funktion differenzieren	13
4.2	Tabellenwerte differenzieren	13
4.3	Einen Spline differenzieren	14
4.4	Welches Verfahren wählen?	14
5	Numerische Integration	15
5.1	Welches Verfahren wähle ich?	15
5.2	Zusammengesetzte Trapez- und Simpson-Regel	15
5.3	Adaptive Simpson mit Diagnosewerten	16
5.4	Romberg-Integration	16
5.5	Fünf-Punkt-Gauß-Legendre	17
5.6	Nicht-endliche Werte und singuläre Integranden	17
5.7	Genauigkeit ist mehr als eine Toleranzzahl	18
6	Matrizen und lineare Gleichungssysteme	19
6.1	1. DenseMatrix	19
6.2	2. Gauß-Elimination	20
6.2.1	Partielle Pivotisierung	20
6.3	3. Wiederverwendbare LU-Faktorisierung	20
6.4	4. Determinanten	21
6.5	5. Inverse Matrix	21
6.6	6. Pivot-Toleranz und numerische Singularität	22
6.7	7. Residuen	22
6.8	8. Gauss-Seidel-Iteration	22
6.9	9. Exceptions gegenüber Iterationsstatus	23
6.10	10. Verfahrensauswahl	23
6.11	11. Numerische Grenzen	24
7	Eigenwerte und Eigenvektoren	25
7.1	Welches Verfahren soll ich verwenden?	25
7.2	Erstes Beispiel mit der Potenzmethode	25
7.3	Eigenvektoren sind nicht eindeutig	26
7.4	Ein Eigenpaar unabhängig prüfen	26
7.5	Potenzmethode genauer betrachtet	27
7.6	Inverse Potenzmethode	27
7.7	Wielandt-Deflation und zweites Eigenpaar	28
7.8	Vollständiges symmetrisches Eigensystem mit Jacobi	28
7.8.1	Was bedeutet das Jacobi-Residual?	29

7.9 Toleranzen sind Abbruchkriterien, keine garantierten Dezimalstellen . .	29
7.10 Umfang und Performance	30
8 Gewöhnliche Differentialgleichungen und Randwertprobleme	31
8.1 Anfangswertprobleme erster Ordnung	31
8.2 RK4 mit fester Schrittweite	31
8.3 Gleichungen zweiter Ordnung mit RK4	31
8.4 Gleichungen n-ter Ordnung mit RK4	32
8.5 Gekoppelte Systeme zweiter Ordnung mit RK4	32
8.6 Lineare Randwertprobleme mit Shooting	32
8.7 Nichtlineare Randwertprobleme mit Shooting	33
8.8 Adaptives RKF45	34
8.9 Adams-Prädiktor-Korrektor	34
8.10 Wahl zwischen den Verfahren	34
8.11 Randresiduum und globale Genauigkeit	35
8.12 Derzeitige Grenzen	35
9 Least-Squares-Approximation	37
9.1 Polynomanpassung	37
9.2 Potenz-, Exponential- und logarithmische Modelle	37
9.3 Fünfgliedrige Fourier-Anpassung	37
9.4 Beliebige lineare Basisfunktionen	38
9.5 Warum QR der bevorzugte Normalfall bleibt	38
9.6 Wann die SVD übernimmt	38
9.7 Residuen und Transformationen verstehen	39
9.8 Praktische Prüfung	39
10 FFT, Faltung und Korrelation	41
10.1 Komplexe FFT	41
10.2 Reelle Eingabe: vollständiges oder kompaktes Spektrum	41
10.3 Reelles Signal rekonstruieren	42
10.4 FFT im Unterschied zur Fourier-Least-Squares-Approximation	42
10.5 Lineare Faltung	42
10.6 Kreuzkorrelation und Lag-Reihenfolge	43
10.7 Leere und ungültige Eingaben	43
10.8 Praktische Grenzen	43
11 Numerische Rezepte, Diagnostik und typische Fehlerfälle	45

11.11. Vier Fragen vor der Übernahme eines numerischen Ergebnisses . . .	45
11.22. Exceptions und Ergebnisstatus haben unterschiedliche Aufgaben . . .	45
11.33. Converged ist eine Beendigungsinformation, kein Exaktheitsbeweis . .	46
11.44. Residuum, Vorwärtsfehler und Fehlerschätzung sind nicht dasselbe .	46
11.4.1 Nullstellensuche	46
11.4.2 Lineare Gleichungssysteme	47
11.4.3 Eigenpaare	47
11.4.4 Adaptive Integration	47
11.4.5 Adaptive ODE-Integration	48
11.55. Eine Toleranz muss zur Problemskala passen	48
11.66. Häufige Beendigungsstatus und sinnvolle Reaktionen	48
11.6.1 MaximumIterationsReached	48
11.6.2 NumericalBreakdown	49
11.6.3 NotBracketed	49
11.6.4 MaximumDepthReached / NumericalResolutionReached	49
11.6.5 MinimumStepSizeReached	49
11.6.6 Singuläre oder numerisch singuläre Matrix	49
11.77. Nicht-endliche Werte sind Fehlerzustände, keine normalen Daten . .	49
11.88. Konditionierung ist etwas anderes als algorithmische Konvergenz . .	50
11.99. Unabhängige Prüfrezepte	50
11.9.1 Nullstelle	50
11.9.2 Direktes lineares Gleichungssystem	50
11.9.3 Eigenpaar	50
11.9.4 Integration	51
11.9.5 ODE	51
11.9.6 Least Squares	51
11.100. Ein praktisches Abnahmemuster	51
11.111. Was protokolliert werden sollte	51
11.122. Kurze Prüfliste	52
12 Moderne dichte lineare Algebra — Erweiterungen 2026	53
12.1 Householder-QR	53
12.2 Cholesky	53
12.3 Singulärwertzerlegung	53
12.3.1 Numerischer Rang ist toleranzabhängig	54
12.3.2 Rangdefizientes Least Squares	54
12.3.3 Pseudoinverse	55

12.3.4	Konditionszahl	55
12.4	Matrixnormen und gemeinsamer Diagnosebericht	55
12.5	Least-Squares-Routing im Toolkit	56
12.6	Verfahrensauswahl	56
12.7	Performance-Perspektive	56
13	Dünnbesetzte lineare Algebra — CSR, CG/PCG, GMRES und BiCGSTAB	57
13.1	Sparse-Matrix aufbauen	57
13.2	Matrix-Vektor-Multiplikation	57
13.3	Dense-Konvertierung und Sparsifizierung	58
13.4	Transponieren	58
13.5	Sparse-Normen	58
13.6	Conjugate Gradient für SPD-Systeme	58
13.6.1	Verifikation des echten Residuums	59
13.7	Preconditioned Conjugate Gradient	59
13.8	Restarted GMRES für allgemeine quadratische Systeme	60
13.8.1	Right-preconditioned GMRES	60
13.9	BiCGSTAB für nichtsymmetrische Systeme mit begrenztem Speicher . .	61
13.9.1	Right-preconditioned BiCGSTAB	61
13.9.2	Auswahl des Sparse-Solvers	61
13.9.3	Solver-Diagnostik	62
13.10	Was die Sparse-Schicht noch nicht liefert	62
A	Historischer Kontext und aktuelle V1-Abdeckung	63
B	Build und Pflege der LaTeX-Ausgabe	65
	Lizenz und Herkunft	67

Erste Schritte mit C#/.NET

1.1 Aktuelle Entwicklungsform

Das SASD Math Toolkit zielt derzeit auf .NET 10 und wird als Klassenbibliothek in diesem Repository entwickelt. Solange noch kein Paket-Feed bereitgestellt wird, kann ein anderes Projekt direkt `src/dotnet/Sasd.Math.Toolkit/Sasd.Math.Toolkit.csproj` referenzieren oder mit der Repository-Solution arbeiten.

Das komplette Repository wird so gebaut und getestet:

```
dotnet restore Sasd.Math.Toolkit.slnx
dotnet build Sasd.Math.Toolkit.slnx --configuration Release
dotnet test Sasd.Math.Toolkit.slnx --configuration Release
```

1.2 Namespaces

Die Assembly heißt `Sasd.Math.Toolkit`; die numerischen APIs beginnen mit `Sasd.Numerics`. Dadurch entsteht keine unnötige Verwechslung mit `System.Math`, und die Fachbereiche bleiben sichtbar:

```
using Sasd.Numerics.Differentiation;
using Sasd.Numerics.Integration;
using Sasd.Numerics.LinearAlgebra;
```

1.3 Eine erste Berechnung

Die erste Ableitung einer aufrufbaren Funktion lässt sich direkt näherungsweise bestimmen:

```
var derivative = NumericalDifferentiation.FirstDerivative(Math.Sin, 0.3);
Console.WriteLine(derivative);
```

Das Ergebnis sollte nahe bei `Math.Cos(0.3)` liegen. Numerische Verfahren liefern Näherungen; Gleitkommaergebnisse sollten daher normalerweise mit Toleranzen und nicht auf exakte Gleichheit verglichen werden.

1.4 Grafische Numerik-Demo starten

Das Repository enthält ein plattformunabhängiges Sample, das echte Toolkit-APIs ausführt und einen eigenständigen HTML-/SVG-Bericht erzeugt:

```
dotnet run --project samples/dotnet/Sasd.Math.Toolkit.Sample/Sasd.Math.Toolkit.  
Sample.csproj
```

Die erzeugte Datei `sasd-math-toolkit-demo.html` kann anschließend in einem Browser geöffnet werden. Der Bericht enthält derzeit Nullstellensuche, Integration, kompakte Real-FFT, Faltung und Korrelation. Netzwerkzugriff und JavaScript werden nicht benötigt, sodass der Bericht zugleich als deterministisches Demo-/Smoke-Artefakt geeignet ist.

Dieses Sample ersetzt die Rolle der historischen Grafikdemos durch neu geschriebenen C#/.NET-Code. Das Rendering bleibt außerhalb der Numerikbibliothek, damit Anwendungsprojekte nicht an eine bestimmte UI- oder Chart-Technologie gebunden werden.

1.5 Fehlerbehandlung

Verletzungen von Programmverträgen wie unpassende Dimensionen, ungültige Indizes oder nicht-endliche Eingaben werden im Allgemeinen über normale .NET-Ausnahmen gemeldet. Iterative mathematisch gültige Verfahren, deren Konvergenz scheitern kann, liefern dagegen typischerweise Ergebnisobjekte mit Status, Iterationszahl und Residuum.

Diese Trennung ist beabsichtigt: Eine fehlerhafte Eingabe ist etwas anderes als eine gültige Iteration, die innerhalb der gesetzten Grenzen nicht konvergiert.

Nullstellen von Gleichungen

Bei der Nullstellensuche wird ein Wert x gesucht, für den

$$f(x) = 0$$

gilt. Das SASD Math Toolkit trennt dabei bewusst drei verwandte Aufgaben: reelle skalare Gleichungen, komplexwertige Gleichungen und die Nullstellensuche bei Polynomen. Die öffentlichen APIs liefern einen Konvergenzstatus, statt so zu tun, als müsse jede mathematisch gültige Iteration zwangsläufig erfolgreich sein.

2.1 Welches Verfahren passt zu welchem Problem?

Situation	Sinnvoller
Eine reelle Nullstelle liegt nachweislich in einem Intervall mit Vorzeichenwechsel	RootSolvers
Gute Startnäherung und Ableitung sind vorhanden	RootSolvers
Keine Ableitung verfügbar, aber zwei brauchbare Startwerte	RootSolvers
Eine Polynomnullstelle und ein brauchbarer Startwert	PolynomialF
Eine allgemeine komplexe Funktion darf die reelle Achse verlassen	ComplexRoot
Eine Polynomnullstelle soll im komplexen Bereich gefunden werden	PolynomialF
Alle Polynomnullstellen werden benötigt	PolynomialF

Es gibt kein universell bestes Verfahren. Eine reelle, geklammerte Nullstelle ist numerisch eine andere Aufgabe als das Bestimmen sämtlicher komplexer Nullstellen eines Polynoms.

2.2 Gemeinsame Optionen und Ergebnisdaten

Die skalaren Verfahren sowie Muller, Newton-Horner und Laguerre verwenden RootFindingOptions:

```
using Sasd.Numerics.RootFinding;

var options = new RootFindingOptions(
    Tolerance: 1e-12,
    MaximumIterations: 100);
```

Tolerance muss positiv sein und wird in der aktuellen Implementierung sowohl für das Residuum als auch für eine relativ skalierte Änderung des Iterationswerts verwendet. MaximumIterations ist eine Sicherheitsgrenze und keine Zusage, dass vorher Konvergenz eintreten muss.

Skalare Verfahren liefern `RootResult`. Besonders wichtig sind:

- `Root`: beste zurückgegebene Nullstellennäherung;
- `FunctionValue`: $f(\text{Root})$ an dieser Stelle;
- `Residual`: $\text{abs}(\text{FunctionValue})$;
- `Iterations`: Anzahl vollständig ausgeführter Iterationen;
- `Status`: Grund für die Beendigung;
- `Converged`: Kurzform für `Status == IterationStatus.Converged`;
- `Message`: optionale Diagnosemeldung.

Komplexe Einzelnullstellenverfahren liefern `ComplexRootResult`; dort ist `Residual` der Betrag $|f(\text{root})|$. `FindAllRootsLaguerre` gibt `PolynomialRootsResult` mit allen gefundenen Nullstellen, Gesamtiterationszahl und maximalem Residuum gegenüber dem ursprünglichen Polynom zurück.

Programmierfehler bzw. Vertragsverletzungen wie NaN-Eingaben oder ein ungültiges Intervall werden als `Exceptions` gemeldet. Erwartbare numerische Ergebnisse wie eine fehlende Bisektionsklammer, eine fast verschwindende Newton-Ableitung oder das Erreichen der Iterationsgrenze erscheinen dagegen im Ergebnisstatus.

2.3 Bisektion: das robuste Intervallverfahren

Bisektion ist die erste Wahl, wenn eine stetige Funktion an den Endpunkten eines Intervalls unterschiedliche Vorzeichen besitzt. Für

$$f(x) = \cos(x) - x$$

klammert $[0, 1]$ die gesuchte Nullstelle:

```
using Sasd.Numerics.RootFinding;

var result = RootSolvers.Bisection(
    x => Math.Cos(x) - x,
    left: 0.0,
    right: 1.0);

if (!result.Converged)
{
    Console.WriteLine($"Bisektion beendet mit {result.Status}: {result.Message}");
}
else
{
    Console.WriteLine($"Nullstelle = {result.Root:G17}");
    Console.WriteLine($"Residuum = {result.Residual:E3}");
}
```

Das Verfahren halbiert das Intervall wiederholt und behält jeweils die Hälfte, deren Endpunkte weiterhin unterschiedliche Vorzeichen besitzen. Der Fortschritt ist dadurch gut vorhersagbar und wesentlich weniger von einem geschickten Startwert abhängig als bei Newton- oder Sekanteniteration.

Haben beide Randwerte dasselbe Vorzeichen, liefert das Toolkit `IterationStatus.NotBracketed`. Es wird weder eine Nullstelle erfunden noch stillschweigend auf ein anderes Verfahren gewechselt. Gleiches Vorzeichen beweist nicht, dass im Intervall keine Nullstelle existiert; es bedeutet lediglich, dass die normale Bisektion keine

gültige Vorzeichenwechselklammer besitzt.

2.4 Newton-Raphson: schnell bei gutem lokalem Modell

Newton-Raphson verwendet die Tangente

$$x(\text{neu}) = x - f(x) / f'(x).$$

Für dieselbe Gleichung:

```
var result = RootSolvers.NewtonRaphson(
    x => Math.Cos(x) - x,
    x => -Math.Sin(x) - 1.0,
    initialGuess: 0.5);
```

In der Nähe einer einfachen Nullstelle konvergiert Newton häufig deutlich schneller als die Bisektion. Das Verfahren ist aber auf brauchbare lokale Ableitungsinformation angewiesen. Ein ungünstiger Startwert kann die Iteration von der gewünschten Nullstelle wegführen, und eine Ableitung nahe null macht den Newton-Schritt instabil. Das SASD Math Toolkit meldet diesen Fall mit `IterationStatus.NumericalBreakdown`, anstatt durch eine fast verschwindende Zahl zu dividieren.

Newton-Raphson ist sinnvoll, wenn die Ableitung zuverlässig und eine plausible Startnäherung vorhanden ist. Soll eine Intervallklammer unbedingt erhalten bleiben, ist Bisektion die sicherere Wahl; ein späteres hybrides Verfahren kann auf diesen Grundbausteinen aufsetzen.

2.5 Sekantenverfahren: lokale Iteration ohne Ableitungsfunktion

Das Sekantenverfahren nähert die Ableitung aus zwei Funktionswerten an:

```
var result = RootSolvers.Secant(
    x => Math.Cos(x) - x,
    firstGuess: 0.0,
    secondGuess: 1.0);
```

Es benötigt keine separate Ableitungsfunktion und konvergiert oft schneller als Bisektion. Anders als bei der Bisektion bilden die beiden Werte jedoch keine dauerhaft geschützte Klammer. Werden die beiden Funktionswerte nahezu gleich, ist die Sekantensteigung numerisch nicht mehr brauchbar; das Ergebnis meldet dann `NumericalBreakdown`.

2.6 Polynome und Horner-Schema

Die Polynom-APIs verwenden `Sasd.Numerics.Polynomials.Polynomial`. Die Koeffizienten werden in **absteigender Potenzreihenfolge** angegeben. Beispiel:

```
using Sasd.Numerics.Polynomials;

// x^3 - 2x - 5
```

```
var polynomial = new Polynomial([1.0, 0.0, -2.0, -5.0]);
```

Diese Reihenfolge ist wichtig. [1, 0, -2, -5] bedeutet nicht „Konstante zuerst“.

`PolynomialRootSolvers.NewtonHorner` verbindet die Newton-Iteration mit erweiterter Horner-Auswertung. Polynomwert und erste Ableitung werden dabei in einem gemeinsamen Durchlauf bestimmt:

```
using System.Numerics;
using Sasd.Numerics.RootFinding;

var result = PolynomialRootSolvers.NewtonHorner(
    polynomial,
    initialGuess: new Complex(2.0, 0.0));
```

Nach einer gefundenen Nullstelle kann `Polynomial.Deflate(root)` den zugehörigen linearen Faktor entfernen. Das Deflationsergebnis enthält Quotientenpolynom und Rest. Ein großer Rest ist ein Hinweis darauf, dass der übergebene Wert keine ausreichend genaue Nullstelle war.

2.7 Muller: von reellen Startwerten in die komplexe Ebene

Das Muller-Verfahren legt durch drei Punkte lokal ein quadratisches Modell. Da die Diskriminante komplex ausgewertet wird, kann eine auf der reellen Achse beginnende Folge von selbst in die komplexe Ebene wechseln.

Für $f(z) = z^2 + 1$:

```
using System.Numerics;

static Complex Function(Complex z) => (z * z) + Complex.One;

var result = ComplexRootSolvers.Muller(
    Function,
    Complex.Zero,
    Complex.One,
    new Complex(2.0, 0.0));
```

Ein konvergiertes Ergebnis sollte nahe $+i$ oder $-i$ liegen. Welche Nullstelle erreicht wird, hängt vom numerischen Weg ab. Ein Einzelnullstellenverfahren garantiert bei mehreren Lösungen nicht ohne zusätzliche Voraussetzungen eine bestimmte davon.

2.8 Laguerre für Polynomnullstellen

Laguerres Verfahren ist auf Polynome zugeschnitten und verwendet Polynomwert sowie erste und zweite Ableitung. Es eignet sich besonders, wenn Nullstellen komplex sein können:

```
var quartic = new Polynomial([1.0, 0.0, 0.0, 0.0, 1.0]); // x^4 + 1
var oneRoot = PolynomialRootSolvers.Laguerre(quartic, Complex.One);
```

Werden alle Nullstellen benötigt, sollte die höhere API verwendet werden, statt selbst eine Deflationsschleife zu schreiben:

```

var polynomial = new Polynomial([1.0, 0.0, 0.0, 0.0, -1.0]); // x^4 - 1
var allRoots = PolynomialRootSolvers.FindAllRootsLaguerre(polynomial);

if (allRoots.Converged)
{
    foreach (var root in allRoots.Roots)
    {
        Console.WriteLine(root);
    }

    Console.WriteLine($"maximales Residuum = {allRoots.MaximumResidual:E3}");
}

```

Die aktuelle All-Roots-Implementierung ist deterministisch. Sie verwendet wiederholte Laguerre-Suchen, synthetische Deflation, feste Ausweichstartwerte auf Basis einer Cauchy-Nullstellenschranke und danach ein Polishing gegenüber dem ursprünglichen Polynom. Dieses Polishing ist wichtig, weil sich Rundungsfehler bei aufeinanderfolgenden Deflationen aufsummieren können.

2.9 Konvergenz richtig interpretieren

Ein kleiner Iterationsschritt und ein kleines Residuum sind verwandte, aber nicht identische Aussagen. Die aktuellen Nullstellensolver akzeptieren Konvergenz, wenn entweder das Funktionsresiduum innerhalb der Toleranz liegt oder die Änderung zwischen zwei Iterationswerten relativ zur Größenordnung ausreichend klein geworden ist. Bei wichtigen Rechnungen sollte deshalb neben Converged immer auch Residual betrachtet werden.

Eine konvergierte Nullstelle ist außerdem nicht automatisch genau die gewünschte Nullstelle. Funktionen können mehrere oder mehrfache Nullstellen besitzen; Startwerte beeinflussen Newton, Sekante, Muller und Laguerre. Die Bisektion bietet eine stärkere Lokalisierung, weil die gesuchte Nullstelle in einem Vorzeichenwechselintervall eingeschlossen bleibt – allerdings nur bei einer gültigen reellen Klammerung.

2.10 Sinnvoller Arbeitsablauf

Zuerst sollte geklärt werden, ob eine allgemeine reelle Gleichung, eine komplexe Gleichung oder ein Polynom vorliegt. Gibt es für ein reelles Problem ein Intervall mit sicherem Vorzeichenwechsel, eignet sich die Bisektion gut als robuste Referenz. Newton oder Sekante sind sinnvoll, wenn Geschwindigkeit wichtiger ist und brauchbare Startinformationen vorhanden sind. Bei Polynomen sollten die spezialisierten Verfahren bevorzugt werden, weil sie Horner-Auswertung, Deflation und All-Roots-Unterstützung wiederverwenden. Für nachvollziehbare numerische Ergebnisse sollten Terminierungsstatus, Residuum und verwendete Toleranzen immer mit dokumentiert werden.

Interpolation

Interpolation konstruiert eine Funktion, die durch bekannte Datenpunkte verläuft. Sie ist sinnvoll, wenn Werte nur an diskreten Stellen bekannt sind, aber dazwischen Werte benötigt werden.

SASD Math Toolkit V1 stellt vier eng verwandte Varianten bereit:

- direkte Lagrange-Polynominterpolation,
- Newton-Interpolation mit dividierten Differenzen,
- natürliche kubische Splines,
- geklemmte bzw. eingespannte kubische Splines.

Alle vier sind Interpolationsverfahren: An den vorgegebenen Stützstellen reproduzieren sie die vorgegebenen Werte bis auf Gleitkomma-Rundung. Sie sind keine Regressionsverfahren und glätten kein Messrauschen.

3.1 Welches Verfahren wählen?

Lagrange eignet sich für kleine Datenmengen und einzelne Auswertungen, wenn die direkte Formel praktisch ist. **Newton mit dividierten Differenzen** ist nützlich, wenn dasselbe Interpolationspolynom konzeptionell mehrfach verwendet oder seine Koeffizientenform benötigt wird. Ein **kubischer Spline** ist meist geeigneter, wenn viele Stützstellen vorliegen und eine stückweise glatte Kurve einem einzigen Polynom hohen Grades vorzuziehen ist.

Natürliche und geklemmte Splines unterscheiden sich durch die Randbedingungen. Beim natürlichen Spline wird die zweite Ableitung an beiden Enden auf null gesetzt. Beim geklemmten Spline werden die bekannten ersten Ableitungen an beiden Enden vorgegeben.

Globale Interpolationspolynome hohen Grades können stark oszillieren, besonders an den Rändern und bei ungünstig verteilten Stützstellen. Mehr Datenpunkte machen ein globales Interpolationspolynom nicht automatisch besser.

3.2 Lagrange-Interpolation

Die Punkte (0,1), (1,4) und (2,9) liegen beispielsweise auf $(x+1)^2$.

```
using Sasd.Numerics.Interpolation;

double[] x = [0.0, 1.0, 2.0];
double[] y = [1.0, 4.0, 9.0];

var value = InterpolationAlgorithms.Lagrange(x, y, 1.5);
Console.WriteLine(value); // ungefähr 6.25
```

Die x-Werte müssen verschieden sein, müssen für die Polynominterpolation aber nicht sortiert sein. Eingabedaten und Auswertungspunkt müssen endlich sein.

Lagrange wertet die Basisformel direkt aus. Dadurch bleibt die Referenzimplementierung leicht nachvollziehbar, führt die $O(n^2)$ -Basisarbeit aber bei jedem Aufruf erneut aus. Für V1 ist diese Verständlichkeit bewusst wichtiger als vorzeitige Optimierung.

3.3 *Newton und dividierte Differenzen*

Dasselbe Interpolationspolynom lässt sich darstellen als

$$a_0 + a_1(x-x_0) + a_2(x-x_0)(x-x_1) + \dots$$

Die Koeffizienten können getrennt berechnet werden:

```
var coefficients = InterpolationAlgorithms.NewtonDividedDifferenceCoefficients(x, y);
```

Für das obige Beispiel sind sie in der durch die x-Reihenfolge definierten Newton-Basis ungefähr [1, 3, 1].

Für eine einzelne Auswertung gibt es die Komfortmethode:

```
var value = InterpolationAlgorithms.NewtonDividedDifference(x, y, 1.5);
```

Das zurückgegebene Koeffizientenarray ist von den Eingabearrays unabhängig. Spätere Änderungen an den Eingaben verändern die bereits berechneten Koeffizienten nicht.

3.4 *Natürlicher kubischer Spline*

Ein kubischer Spline verwendet auf jedem Intervall ein eigenes kubisches Polynom und erzwingt an inneren Stützstellen Stetigkeit der Funktion sowie ihrer ersten und zweiten Ableitung.

```
double[] x = [0.0, 1.0, 2.0, 3.0];  
double[] y = [0.0, 1.0, 0.0, 1.0];  
  
var spline = InterpolationAlgorithms.NaturalCubicSpline(x, y);  
  
var value = spline.Evaluate(1.5);  
var slope = spline.FirstDerivative(1.5);  
var curvature = spline.SecondDerivative(1.5);
```

Der natürliche Spline ergänzt die Randbedingungen

$$S''(x_0) = 0 \text{ und } S''(x_n) = 0.$$

Das ist eine neutrale mathematische Randannahme und keine Behauptung, dass die reale zugrunde liegende Funktion dort tatsächlich keine Krümmung besitzt.

3.5 Geklemmter kubischer Spline

Sind die Steigungen an den beiden Rändern bekannt, können sie vorgegeben werden:

```
static double F(double x) => x * x * x;
static double Df(double x) => 3.0 * x * x;

double[] x = [0.0, 1.0, 2.0, 3.0];
var y = x.Select(F).ToArray();

var spline = InterpolationAlgorithms.ClampedCubicSpline(
    x,
    y,
    leftDerivative: Df(x[0]),
    rightDerivative: Df(x[^1]));
```

Mit exakten Randableitungen reproduziert ein geklemmter kubischer Spline ein kubisches Polynom bis auf Rundungsfehler exakt. Bei Messdaten sollten Randableitungen nur vorgegeben werden, wenn sie fachlich sinnvoll bekannt sind; ungenaue Steigungsbedingungen können gerade den Randbereich verzerren.

3.6 Definitionsintervall und Unveränderlichkeit

Spline-Stützstellen müssen streng aufsteigend sein. Das Spline-Objekt stellt zur Verfügung:

```
spline.Knots
spline.IntervalStart
spline.IntervalEnd
spline.SegmentCount
```

Knots ist schreibgeschützt, und der Spline besitzt seine internen Koeffizientendaten selbst. Er kann nach der Konstruktion also wiederverwendet werden, ohne durch spätere Änderungen an Eingabearrays verändert zu werden.

Spline-Auswertung extrapoliert bewusst **nicht**. Aufrufe außerhalb von [IntervalStart, IntervalEnd] sowie NaN oder Unendlich führen zu `ArgumentOutOfRangeException`. Wenn Extrapolation gewünscht ist, sollte sie eine ausdrückliche Modellierungsentscheidung sein und kein unbeabsichtigter Nebeneffekt der Interpolation.

Die globalen Verfahren Lagrange und `NewtonDividedDifference` können dagegen mathematisch auch außerhalb des gegebenen x-Bereichs ausgewertet werden. Das bedeutet nicht, dass eine solche Polynomextrapolation zuverlässig ist.

3.7 Ungültige Daten und numerische Probleme

Doppelte x-Werte sind bei der Polynominterpolation ungültig, weil sonst durch null dividiert werden müsste. Bei Splines müssen die Stützstellen zusätzlich sortiert und streng aufsteigend sein. Nicht-endliche Eingangsdaten werden abgelehnt.

Auch endliche Daten können so extrem skaliert sein, dass Zwischenrechnungen überlaufen oder das Spline-Gleichungssystem numerisch unbrauchbar wird. In solchen

Fällen wirft die Referenzimplementierung `ArithmeticException`, statt still NaN oder Unendlich zurückzugeben.

Bei schwierigen Datensätzen sollte man zunächst die Größenordnungen prüfen, x-Werte gegebenenfalls mathematisch sinnvoll verschieben oder normieren und unnötig hohe globale Polynomgrade vermeiden.

3.8 Interpolation ist keine Approximation

Interpolation zwingt die resultierende Kurve durch jeden vorgegebenen Wert. Least-Squares-Approximation erlaubt dagegen bewusst Residuen, um ein Modell an veräuschte oder überbestimmte Daten anzupassen. Die Wahl sollte von der Bedeutung der Daten abhängen und nicht nur davon, welche Kurve optisch glatt aussieht.

Numerische Differentiation

Numerische Differentiation schätzt Ableitungen, wenn keine analytische Ableitung verfügbar oder praktisch nutzbar ist. Das SASD Math Toolkit bietet dafür drei Wege: eine aufrufbare Funktion differenzieren, Tabellenwerte differenzieren oder einen aus Tabellenwerten erzeugten kubischen Spline differenzieren.

4.1 Eine Funktion differenzieren

```
using Sasd.Numerics.Differentiation;  
  
var first = NumericalDifferentiation.FirstDerivative(Math.Sin, 0.3);  
var second = NumericalDifferentiation.SecondDerivative(Math.Sin, 0.3);
```

Die Standardroutinen verwenden zentrale Differenzen mit Richardson-Verfeinerung. Zusätzlich stehen explizite Fünfpunktverfahren zur Verfügung.

4.2 Tabellenwerte differenzieren

Angenommen, eine Messreihe enthält steigende x-Werte und dazugehörige y-Werte:

```
var x = new[] { 0.0, 0.5, 1.1, 2.0, 3.0 };  
var y = x.Select(value => value * value).ToArray();  
  
var first = TabularDifferentiation.FirstDerivativeThreePoint(x, y, 2);  
var second = TabularDifferentiation.SecondDerivativeFivePoint(x, y, 2);
```

Das letzte Argument ist der Index des Tabellenwertes, an dem die Ableitung gesucht wird. Die x-Werte dürfen ungleichmäßig verteilt sein, müssen aber endlich und streng steigend sein.

Zur Verfügung stehen Zwei-, Drei- und Fünfpunktverfahren für die erste sowie Drei- und Fünfpunktverfahren für die zweite Ableitung.

Am Anfang oder Ende der Tabelle verschiebt die Bibliothek den lokalen Stützstellenbereich automatisch, statt außerhalb der Daten zuzugreifen. Bei glatten Daten kann eine größere Punktzahl den Diskretisierungsfehler verringern. Bei verrauschten Messungen verstärkt Differentiation jedoch häufig das Rauschen; mehr Stützstellen sind deshalb nicht automatisch besser.

4.3 Einen Spline differenzieren

Ein Spline ist sinnvoll, wenn die Daten zunächst durch eine glatte stückweise kubische Kurve beschrieben werden sollen. Für wiederholte Auswertungen wird der Spline einmal erzeugt:

```
using Sasd.Numerics.Interpolation;

var spline = InterpolationAlgorithms.NaturalCubicSpline(x, y);
var slope = spline.FirstDerivative(1.4);
var curvature = spline.SecondDerivative(1.4);
```

Für eine einzelne Berechnung gibt es Komfortfunktionen:

```
var slope = SplineDifferentiation.NaturalFirstDerivative(x, y, 1.4);
```

Sind die Randableitungen bekannt, kann ein geklemmter Spline verwendet werden:

```
var slope = SplineDifferentiation.ClampedFirstDerivative(
    x,
    y,
    leftDerivative: 0.0,
    rightDerivative: 6.0,
    point: 1.4);
```

Spline-Auswertungen sind auf das Interpolationsintervall beschränkt. Das SASD Math Toolkit extrapoliert nicht still über die bereitgestellten Daten hinaus.

4.4 Welches Verfahren wählen?

Direkte Funktionsdifferentiation eignet sich, wenn die Funktion in der Umgebung des Zielpunkts frei ausgewertet werden kann. Tabellarische Differentiation passt, wenn die vorhandenen Datenpunkte selbst untersucht werden. Spline-Differentiation ist sinnvoll, wenn eine glatte Interpolationskurve ein vertretbares Modell darstellt und Ableitungen an beliebigen Stellen innerhalb des Intervalls benötigt werden.

Kein Verfahren kann Information rekonstruieren, die in den Daten nicht enthalten ist. Ableitungen aus verrauschten Messwerten müssen deshalb vorsichtiger interpretiert werden als Ableitungen glatter mathematischer Funktionen.

Numerische Integration

Numerische Integration nähert ein bestimmtes Integral an, wenn eine Stammfunktion nicht verfügbar, unpraktisch oder nur als ausführbarer Code gegeben ist. Das SASD Math Toolkit bietet feste Newton-Cotes-Verfahren, Romberg-Extrapolation, die Fünf-Punkt-Gauß-Legendre-Quadratur sowie adaptive Varianten.

```
using Sasd.Numerics.Integration;
```

Alle aktuellen eindimensionalen Verfahren verlangen endliche Grenzen mit $a < b$. Wird das Integral in Gegenrichtung benötigt, sollten die Grenzen bewusst vertauscht und das Ergebnis negiert werden.

5.1 Welches Verfahren wähle ich?

Die **zusammengesetzte Trapezregel** eignet sich als einfacher Referenzansatz oder wenn ein stückweise lineares Verhalten naheliegt. **Composite Simpson** ist für glatte Funktionen attraktiv, wenn eine feste gerade Teilintervallzahl in Ordnung ist. **Adaptive Simpson** ist sinnvoll, wenn sich die Schwierigkeit innerhalb des Integrationsintervalls stark ändert und die Verfeinerung dort stattfinden soll, wo sie benötigt wird.

Romberg eignet sich für glatte Funktionen, bei denen wiederholte Trapezverfeinerung und Richardson-Extrapolation gut wirken. **Fünf-Punkt-Gauß-Legendre** ist ein hochgradiges Verfahren für ein einzelnes Intervall; in exakter Arithmetik werden Polynome bis Grad neun exakt integriert. Die **adaptive Gauß-Legendre-Variante** ist sinnvoll, wenn ein einzelnes Gauß-Panel für das gesamte Intervall nicht genügt.

5.2 Zusammengesetzte Trapez- und Simpson-Regel

```
var trapezoid = NumericalIntegration.CompositeTrapezoid(  
    Math.Sin,  
    0.0,  
    Math.PI,  
    intervals: 200);  
  
var simpson = NumericalIntegration.CompositeSimpson(  
    Math.Sin,  
    0.0,  
    Math.PI,  
    intervals: 200);
```

Die Teilintervallzahl gehört zum numerischen Modell. Mehr Panels reduzieren bei hinreichend glatten Funktionen meist den Diskretisierungsfehler, erhöhen aber die Zahl der Funktionsauswertungen und stoßen irgendwann an Gleitkommagrenzen. Für

Composite Simpson muss intervals gerade sein.

Als Plausibilitätsprüfung sind die Exaktheitseigenschaften hilfreich: Die Trapezregel ist für lineare Funktionen exakt, Simpson in exakter Arithmetik für kubische Polynome.

5.3 Adaptive Simpson mit Diagnosewerten

Die einfache API liefert nur den besten Schätzwert:

```
var integral = NumericalIntegration.AdaptiveSimpson(  
    Math.Sin,  
    0.0,  
    Math.PI,  
    tolerance: 1e-10,  
    maximumDepth: 20);
```

Wenn eine Anwendung unterscheiden muss, ob die Toleranz wirklich erfüllt oder nur eine Grenze erreicht wurde, sollte die detaillierte API verwendet werden:

```
var result = NumericalIntegration.AdaptiveSimpsonDetailed(  
    Math.Sin,  
    0.0,  
    Math.PI,  
    tolerance: 1e-10,  
    maximumDepth: 20);  
  
if (!result.Converged)  
{  
    Console.WriteLine($"Abbruchstatus: {result.Status}");  
}  
  
Console.WriteLine(result.Value);  
Console.WriteLine(result.EstimatedError);  
Console.WriteLine(result.FunctionEvaluations);
```

`AdaptiveIntegrationStatus.Converged` bedeutet, dass jedes akzeptierte Panel sein lokales Kriterium erfüllt hat. `MaximumDepthReached` bedeutet, dass mindestens ein Bereich noch weiter verfeinert werden sollte, als die konfigurierte Rekursionstiefe zulässt. `NumericalResolutionReached` bedeutet, dass sich ein Intervall mit `double` nicht mehr in einen unterschiedlichen Mittelpunkt teilen ließ. In beiden nicht konvergierten Fällen bleibt `Value` der beste verfügbare Schätzwert, darf aber nicht so dargestellt werden, als sei die gewünschte Toleranz garantiert.

Der Simpson-Fehlerindikator basiert auf der Differenz zwischen einem Simpson-Panel und seinen beiden Halbintervallen samt Richardson-Korrektur. Er ist eine Schätzung und kein mathematischer Beweis für den tatsächlichen Fehler.

5.4 Romberg-Integration

Romberg baut aus sukzessiv halbierten Trapezregeln eine Extrapolationstabelle auf:

```
var result = NumericalIntegration.RombergDetailed(  
    x => Math.Exp(-x * x),  
    0.0,
```

```
1.0,
tolerance: 1e-10,
maximumLevels: 12);
```

result.Value enthält die beste diagonale Extrapolation. EstimatedError ist der Absolutbetrag der Differenz der letzten beiden Diagonaleinträge. Status ist IterationStatus. Converged, wenn diese Differenz die gewünschte Toleranz erfüllt; sonst wird der letzte verfügbare Wert mit MaximumIterationsReached zurückgegeben.

Der Aufwand verdoppelt sich ungefähr mit jeder zusätzlichen Trapezstufe. Die Referenzimplementierung begrenzt maximumLevels deshalb auf 30, damit das ganzzahlige Unterteilungsmodell definiert bleibt. Werte in der Nähe dieser Obergrenze können trotzdem sehr teuer sein; normale Anwendungen sollten deutlich weniger Stufen verwenden.

5.5 Fünf-Punkt-Gauß-Legendre

Die feste Fünf-Knoten-Regel kann direkt verwendet werden:

```
var integral = NumericalIntegration.GaussLegendre5(
    x => Math.Exp(x),
    0.0,
    1.0);
```

Knoten und Gewichte werden vom Standardintervall $[-1, 1]$ auf $[a, b]$ abgebildet. Bei glatten Funktionen kann Gauß-Quadratur mit vergleichsweise wenigen Funktionsauswertungen eine hohe Genauigkeit erreichen.

Die adaptive Variante vergleicht eine Fünf-Punkt-Regel über das gesamte Panel mit der Summe zweier Fünf-Punkt-Regeln über die beiden Hälften:

```
var result = NumericalIntegration.AdaptiveGaussLegendre5Detailed(
    Math.Exp,
    0.0,
    1.0,
    tolerance: 1e-12,
    maximumDepth: 16);
```

EstimatedError ist hier die aufsummierte Verfeinerungsdifferenz, die der Algorithmus als Kriterium verwendet. Auch sie ist ein Fehlerindikator und keine formale Schranke.

5.6 Nicht-endliche Werte und singuläre Integranden

Alle Integrationsverfahren weisen NaN und Unendlich zurück, wenn der Integrand solche Werte liefert. Das ist beabsichtigt: Ein nicht-endlicher Einzelwert soll nicht still durch eine lange Quadratur fortgepflanzt werden.

Uneigentliche Integrale oder Funktionen mit Rand-Singularitäten benötigen daher eine explizite Transformation oder Intervallbehandlung durch die Anwendung. Die aktuellen V1-Routinen sind Verfahren für endliche Intervalle; unendliche Grenzen oder Hauptwerte werden nicht automatisch interpretiert.

5.7 Genauigkeit ist mehr als eine Toleranzzahl

Eine angegebene Toleranz steuert nur das interne Verfeinerungskriterium. Sie kann keine schlecht skalierte Funktion, Sprungstellen, nicht aufgelöste schmale Spitzen, Auslöschung oder Präzisionsverlust im Benutzer-Callback reparieren. Bei wichtigen Ergebnissen sollte man Methode oder numerische Parameter variieren und prüfen, ob das Ergebnis stabil bleibt.

Die detaillierten adaptiven und Romberg-Ergebnisobjekte existieren deshalb bewusst: Anwendungen sollen nachvollziehen können, wie ein Wert zustande kam, statt nur einen nackten `double` zu speichern.

Matrizen und lineare Gleichungssysteme

Viele numerische Verfahren führen letztlich auf ein lineares Gleichungssystem

$$A * x = b$$

mit der Koeffizientenmatrix A , dem unbekannten Vektor x und der rechten Seite b . Das SASD Math Toolkit stellt dafür eine bewusst kleine dichte Matrixbasis sowie direkte und iterative Lösungsverfahren für den V1-Referenz- und Lernumfang bereit.

Die Implementierung soll zuerst verständlich sein. Sie ist kein Ersatz für einen großen Sparse-Solver oder eine optimierte BLAS/LAPACK-Umgebung.

6.1 1. DenseMatrix

Eine Matrix kann aus einem rechteckigen C#-Array erzeugt werden:

```
using Sasd.Numerics.LinearAlgebra;

var matrix = new DenseMatrix(new double[,]
{
    { 4.0, 1.0 },
    { 2.0, 3.0 }
});

Console.WriteLine(matrix.Rows);    // 2
Console.WriteLine(matrix.Columns); // 2
Console.WriteLine(matrix.IsSquare); // true
```

Einträge sind veränderlich:

```
matrix[0, 1] = 1.5;
```

die Dimensionen dagegen nach der Konstruktion fest. Matrixeinträge müssen endlich sein. NaN und Unendlich werden abgewiesen, weil sie sonst nachfolgende numerische Rechnungen still kontaminieren würden.

Clone() erzeugt eine unabhängige Kopie. Matrix-Vektor- und Matrix-Matrix-Multiplikation stehen ebenfalls zur Verfügung:

```
var y = matrix.Multiply(new[] { 1.0, 2.0 });
var product = matrix.Multiply(DenseMatrix.Identity(2));
```

Die Multiplikation verwendet bewusst einfache Schleifen. Bei arithmetischem Überlauf

wird eine Ausnahme ausgelöst, statt eine Matrix mit unendlichen Werten zurückzugeben.

6.2 2. Gauß-Elimination

Für ein einzelnes kleines dichtes System ist `SolveGaussian` das direkte klassische Verfahren:

```
var a = new DenseMatrix(new double[,]  
{  
    { 2.0, 1.0 },  
    { 5.0, 7.0 }  
});  
  
var b = new[] { 11.0, 13.0 };  
var x = LinearSystemSolvers.SolveGaussian(a, b);
```

Die Methode kopiert `A` und `b`; die Objekte des Aufrufers werden nicht verändert.

6.2.1 Partielle Pivotisierung

Partielle Pivotisierung ist standardmäßig aktiv und sollte normalerweise aktiv bleiben. In jedem Eliminationsschritt wird der betragsmäßig größte verfügbare Eintrag der aktuellen Pivotspalte gewählt.

Eine reguläre Matrix kann beispielsweise mit einer Null auf der ersten Diagonalposition beginnen:

```
0  2  
1  3
```

Eine Gauß-Elimination ohne Pivotisierung kann den ersten Diagonaleintrag nicht verwenden. Mit partieller Pivotisierung werden die Zeilen vertauscht und die Rechnung kann normal fortgesetzt werden.

`partialPivoting: false` bleibt für historischen und didaktischen Vergleich erhalten, nicht weil dies für Anwendungen empfohlen wäre.

6.3 3. Wiederverwendbare LU-Faktorisierung

Wenn mehrere Systeme dieselbe Koeffizientenmatrix verwenden, sollte die Elimination nicht jedes Mal wiederholt werden. Die Matrix wird einmal faktorisiert:

```
var lu = LinearSystemSolvers.FactorizeLu(a);  
  
var x1 = lu.Solve(new[] { 11.0, 13.0 });  
var x2 = lu.Solve(new[] { 4.0, 9.0 });
```

Es gilt die Konvention

$$P * A = L * U$$

mit der Permutation P , der unteren Dreiecksmatrix L mit Einsen auf der Diagonale und der oberen Dreiecksmatrix U .

Für Kontrolle und Diagnose stehen zur Verfügung:

```
var lower = lu.LowerTriangular;
var upper = lu.UpperTriangular;
var permutation = lu.Permutation;
var toleranceUsed = lu.PivotTolerance;
```

Die öffentlich gelieferten Matrizen bzw. Arrays sind unabhängige Kopien, wenn eine Veränderung sonst die Faktorisierung beschädigen könnte.

Mehrere rechte Seiten können als Spalten einer `DenseMatrix` übergeben werden:

```
var bMany = DenseMatrix.Identity(2);
var xMany = lu.Solve(bMany);
```

Genau dieses Muster wird auch zum Aufbau einer Inversen verwendet.

6.4 4. Determinanten

Die Determinante kann direkt berechnet werden:

```
var determinant = LinearSystemSolvers.Determinant(a);
```

oder aus einer bereits vorhandenen LU-Faktorisierung:

```
var determinant = lu.Determinant();
```

Existiert die Faktorisierung bereits, ist der LU-Weg sinnvoller, weil die Diagonale von U und die Parität der Zeilenvertauschungen wiederverwendet werden.

Eine Determinante nahe null kann auf Singularität hinweisen, ist aber allein kein zuverlässiges Maß für die Kondition einer Matrix. Ihr Betrag hängt außerdem stark von der Skalierung ab.

6.5 5. Inverse Matrix

Das Toolkit bietet:

```
var inverse = LinearSystemSolvers.Inverse(a);
```

Intern wird genau eine LU-Faktorisierung erzeugt und gegen sämtliche Spalten der Einheitsmatrix gelöst.

Wenn das eigentliche Ziel jedoch $A \cdot x = b$ ist, sind `SolveGaussian` oder `LuFactorization`.`Solve` normalerweise besser. Die explizite Bildung von A^{-1} benötigt mehr Arbeit und kann numerische Fehler verstärken, ohne zusätzliche Information zu liefern.

Eine Inverse sollte verwendet werden, wenn ein Algorithmus wirklich die inverse Matrix benötigt – nicht nur, weil die symbolische Formel $x = A^{-1} \cdot b$ lautet.

6.6 6. Pivot-Toleranz und numerische Singularität

Die direkten Verfahren verwenden eine positive absolute `pivotTolerance`. Ein Pivotkandidat mit einem Betrag kleiner oder gleich diesem Grenzwert wird als numerisch null behandelt.

```
var lu = LuFactorization.Decompose(a, pivotTolerance: 1e-12);
```

Das ist eine praktische Schutzschwelle, aber keine vollständige Konditionsanalyse. Eine Matrix kann Pivots oberhalb der Schwelle besitzen und dennoch schlecht konditioniert sein.

Bei unterschiedlich skalierten Problemfamilien muss dieselbe absolute Toleranz nicht gleich sinnvoll sein. V1 hält die Regel bewusst einfach und explizit; spätere SASD-Ausbaustufen können Skalierung und Konditionsschätzung ergänzen, ohne das grundlegende Lösungsmodell zu ändern.

6.7 7. Residuen

Nach dem Lösen sollte geprüft werden, wie gut der berechnete Vektor die tatsächlich übergebenen Gleichungen erfüllt:

```
var residual = LinearSystemSolvers.ResidualInfinityNorm(a, x, b);
```

Berechnet wird

```
||A*x - b|| = max_i |(A*x-b)_i|
```

Ein kleines Residuum ist ein wichtiges Indiz, beweist bei einer schlecht konditionierten Matrix aber nicht, dass x nahe an der exakten Lösung liegt.

Die Unterscheidung ist zentral:

- **Residuum:** Wie gut erfüllt der berechnete Vektor die angegebenen Gleichungen?
- **Vorwärtsfehler:** Wie nahe liegt der Vektor an der exakten mathematischen Lösung?
- **Kondition:** Wie empfindlich reagiert die Lösung auf kleine Änderungen der Eingangsdaten?

V1 stellt das Residuum bereit, aber noch keinen Konditionsschätzer.

6.8 8. Gauss-Seidel-Iteration

Gauss-Seidel ist eine iterative Alternative:

```
var result = LinearSystemSolvers.GaussSeidel(  
    new DenseMatrix(new double[,]  
    {  
        { 4.0, 1.0 },  
        { 2.0, 3.0 }  
    }  
),  
    new[] { 1.0, 2.0 },  
    tolerance: 1e-12,
```

```

        maximumIterations: 200);

if (result.Converged)
{
    Console.WriteLine(result.Value[0]);
    Console.WriteLine(result.Residual);
}

```

Konvergenz wird erst gemeldet, wenn sowohl die größte Änderung einer Komponente als auch die Unendlichkeitsnorm des Residuums innerhalb der verlangten Toleranz liegen.

Mögliche Statuswerte sind unter anderem:

- Converged – die V1-Konvergenzkriterien wurden erfüllt;
- MaximumIterationsReached – das Iterationsbudget ist aufgebraucht;
- NumericalBreakdown – beispielsweise bei einer Null bzw. nahezu Null auf der Diagonale oder bei numerischem Überlauf.

Gauss-Seidel **konvergiert nicht für jede reguläre Matrix**. Strikte Diagonaldominanz ist eine verbreitete hinreichende, aber keine notwendige Bedingung. Soll einfach ein kleines beliebiges dichtes System gelöst werden, ist pivotierte LU normalerweise der sicherere Ausgangspunkt.

6.9 9. Exceptions gegenüber Iterationsstatus

Die lineare Algebra folgt derselben SASD-Konvention wie die übrigen Bereiche.

Ungültige Verträge wie falsche Dimensionen, nicht-endliche Eingangswerte oder eine nicht-positive Toleranz führen zu Exceptions. Auch eine direkte Faktorisierung, die mangels brauchbarem Pivot nicht fortgesetzt werden kann, wirft eine `ArithmeticException`.

Gauss-Seidel ist dagegen ein iterativer Prozess. Erwartbare iterative Ergebnisse wie das Erreichen der Maximalzahl von Iterationen oder ein numerischer Zusammenbruch werden über `IterativeResult<double[]>` und `IterationStatus` dargestellt.

6.10 10. Verfahrensauswahl

Eine praktische V1-Faustregel lautet:

Situation	Sinnvoller Ausgangspunkt
Ein kleines dichtes System	Pivotiertes <code>SolveGaussian</code>
Viele rechte Seiten mit demselben A	<code>FactorizeLu</code> , danach wiederholtes <code>Solve</code>
Determinante bei bereits vorhandener LU	<code>lu.Determinant()</code>
Die inverse Matrix selbst wird benötigt	<code>lu.Inverse()</code> / <code>LinearSystemSolvers.Inverse</code>
Iteratives Experiment mit geeigneter Matrix	<code>GaussSeidel</code>
Großes/Sparse/Hochleistungsproblem	Spezialisierter Backend statt V1-Referenzmatrix

6.11 11. Numerische Grenzen

Lineare Gleichungssysteme zeigen besonders deutlich, weshalb „das Programm hat eine Zahl geliefert“ nicht gleichbedeutend mit „das Problem ist numerisch sicher“ ist.

Zu beachten sind insbesondere:

- schlecht skalierte Zeilen oder Spalten;
- nahezu linear abhängige Gleichungen;
- sehr unterschiedliche Größenordnungen der Koeffizienten;
- ein kleines Residuum, das wegen der Gesamtskalierung des Problems allein wenig aussagekräftig ist;
- iterative Verfahren, deren Konvergenzverhalten nicht geprüft wurde.

Die V1-Implementierung stellt bewusst partielle Pivotisierung, explizite Pivotschwellen, Residuen und Fehlerdiagnosen bereit und bleibt gleichzeitig klein genug, um sie nachvollziehen zu können. Konditionsschätzung, Sparse-Speicherung und Hochleistungskerne gehören in spätere Meilensteine und werden nicht hinter einer scheinbar einfachen V1-API versteckt.

Eigenwerte und Eigenvektoren

Bei einem Eigenwertproblem werden ein Skalar λ und ein von Null verschiedener Vektor v gesucht, so dass

$$A * v = \lambda * v$$

gilt. Solche Probleme treten unter anderem bei Stabilitätsanalysen, Schwingungsformen, wiederholten linearen Transformationen und Hauptrichtungen auf. SASD Math Toolkit V1 enthält mehrere klassische Verfahren für reelle dichte Matrizen. Sie lösen unterschiedliche Aufgaben; es gibt nicht den einen Eigenwertsolver, der für jede Matrix die beste Wahl ist.

7.1 Welches Verfahren soll ich verwenden?

Verwende **PowerMethod**, wenn ein Eigenpaar zum betragsmäßig größten Eigenwert gesucht wird und das Spektrum eine hinreichend klare dominante Richtung besitzt.

Verwende **InversePowerMethod**, wenn der Eigenwert nahe Null gesucht wird und die Matrix nicht singulär ist. Das V1-Verfahren arbeitet unverschoben und wählt nicht heimlich einen Shift.

Verwende **WielandtSecondEigenpair** vor allem für den klassischen historischen Ablauf zum Eigenwert mit dem zweitgrößten Betrag. Es ist ein wertvolles Referenzverfahren, aber mehrfache bzw. nahezu mehrfache dominante Eigenwerte sind eine bekannte Schwachstelle.

Verwende **CyclicJacobi**, wenn die Matrix reell und symmetrisch ist und das vollständige Eigensystem benötigt wird. Für kleine dichte symmetrische Probleme ist dies häufig die informativste V1-API, weil alle Eigenwerte und eine orthonormale Eigenvektorbasis zurückgegeben werden.

V1 enthält noch keinen allgemeinen QR-/Schur-Eigensolver für nicht-symmetrische Matrizen und keine allgemeine API für komplexe Eigenwerte.

7.2 Erstes Beispiel mit der Potenzmethode

```
using Sasd.Numerics.LinearAlgebra;

var matrix = new DenseMatrix(new double[,]
{
    { 4.0, 1.0, 0.0 },
    { 1.0, 3.0, 0.0 },
    { 0.0, 0.0, 1.0 }
});

var result = EigenSolvers.PowerMethod(
```

```
matrix,
tolerance: 1e-12,
maximumIterations: 500);

if (result.Converged)
{
    var pair = result.Value;
    Console.WriteLine($"lambda = {pair.Eigenvalue}");
    Console.WriteLine($"Residuum = {result.Residual}");

    foreach (var component in pair.Components)
    {
        Console.WriteLine(component);
    }
}
else
{
    Console.WriteLine($"{{result.Status}}: {{result.Message}}");
}
```

Das Ergebnis besteht bewusst nicht nur aus einer Zahl. `IterativeResult<Eigenpair>` dokumentiert, wie das Verfahren beendet wurde:

- `Converged` bedeutet, dass die Konvergenzkriterien des Verfahrens erfüllt wurden;
- `Iterations` enthält die Zahl der verwendeten Iterationen;
- `Residual` ist bei Potenz- und inverser Potenzmethode der euklidische Defekt der Eigenwertgleichung;
- `MaximumIterationsReached` liefert die letzte Näherung, kennzeichnet sie aber ausdrücklich als nicht bis zum geforderten Kriterium konvergiert;
- `NumericalBreakdown` kennzeichnet einen numerischen Zustand, in dem die Iteration nicht sinnvoll fortgesetzt werden konnte.

Ungültige Methodenparameter werden als Argument-Ausnahmen behandelt und nicht als normale Iterationsstatuswerte.

7.3 *Eigenvektoren sind nicht eindeutig*

Ist v ein Eigenvektor, dann ist jedes von Null verschiedene skalare Vielfache ebenfalls ein Eigenvektor. Insbesondere beschreiben v und $-v$ dieselbe Eigenrichtung. Die SASD-Solver normalisieren ihre Ergebnisvektoren, aber ein Test sollte normalerweise nicht nur deshalb fehlschlagen, weil eine andere mathematisch korrekte Implementierung das umgekehrte Vorzeichen gewählt hat.

Bei einem mehrfachen Eigenwert sind sogar die einzelnen Eigenvektoren innerhalb des zugehörigen Eigenraums nicht eindeutig. Ein vollständiger symmetrischer Eigensolver darf eine andere orthonormale Basis desselben Unterraums liefern und trotzdem korrekt sein.

7.4 *Ein Eigenpaar unabhängig prüfen*

Für ein Paar (λ, v) misst

$$||A \cdot v - \lambda \cdot v||^2$$

wie gut die definierende Gleichung erfüllt ist. SASD stellt diese Diagnose direkt bereit:

```
var residual = EigenSolvers.EigenpairResidualNorm(matrix, result.Value);
```

Dies ist eine sehr nützliche Regressions- und Plausibilitätsprüfung. Ein kleines Residuum bedeutet jedoch **nicht**, dass das Eigenwertproblem gut konditioniert ist. Eng zusammenliegende Eigenwerte können empfindlich sein, obwohl ein berechnetes Paar die Gleichung mit kleinem Defekt erfüllt.

Eigenpair besitzt seine Vektordaten selbst. Components ist schreibgeschützt; Eigenvector liefert eine defensive Array-Kopie. Änderungen an dieser Kopie verändern das numerische Ergebnis nicht.

7.5 Potenzmethode genauer betrachtet

Die Potenziteration multipliziert den aktuellen Vektor wiederholt mit der Matrix und normalisiert das Ergebnis. Unter passenden Voraussetzungen verlieren die Anteile zu betragsmäßig kleineren Eigenwerten relativ an Bedeutung, so dass sich die dominante Eigenrichtung durchsetzt.

Daraus folgt unmittelbar eine Einschränkung: Der Startvektor muss einen von Null verschiedenen Anteil in der gesuchten Eigenrichtung besitzen. SASD verwendet standardmäßig deterministisch den Einsvektor. Ist bekannt, dass dieser zur gesuchten Richtung orthogonal sein könnte, sollte ein eigener Startvektor angegeben werden:

```
var result = EigenSolvers.PowerMethod(
    matrix,
    initialVector: new[] { 1.0, 0.2, -0.4 },
    tolerance: 1e-12);
```

Ein kleiner spektraler Abstand zwischen dem größten und zweitgrößten Betrag bedeutet häufig langsamere Konvergenz. Sind zwei dominante Beträge gleich, kann die einfache Potenzmethode ungeeignet sein.

Die Implementierung normalisiert skaliert. Deshalb scheitert ein sehr großer, aber endlicher Startvektor nicht allein daran, dass seine Komponenten bei einer naiven Normberechnung quadriert würden.

7.6 Inverse Potenzmethode

Die unverschobene inverse Iteration löst wiederholt

$$A * y = x$$

und normalisiert y . Eigenrichtungen zu kleinen Eigenwertbeträgen werden durch die inverse Operation verstärkt. SASD faktorisiert A einmal per LU mit partieller Pivotisierung und verwendet diese Faktorisierung für alle Iterationen erneut.

```
var smallest = EigenSolvers.InversePowerMethod(
    matrix,
    tolerance: 1e-12,
    maximumIterations: 200,
    pivotTolerance: 1e-14);
```

`pivotTolerance` gehört zur LU-Faktorisierung: Pivots bis zu dieser Schwelle werden als numerisch singulär behandelt. Dieser Wert ist nicht mit der Eigenwert-Konvergenztoleranz zu verwechseln.

Da die V1-Methode unverschoben ist, kann eine singuläre Matrix mit dieser API nicht behandelt werden. Die LU-Faktorisierung bricht bereits vor Beginn der Iteration ab. Eine spätere verschobene inverse Iteration sollte einen ausdrücklichen `shift-Parameter` erhalten.

7.7 *Wielandt-Deflation und zweites Eigenpaar*

Die klassische Wielandt-Transformation entfernt über ein Rang-1-Update einen bereits bekannten Eigenwert. Die direkte API ist sinnvoll, wenn bereits ein approximiertes Eigenpaar vorliegt:

```
var dominant = EigenSolvers.PowerMethod(matrix, tolerance: 1e-12);
if (!dominant.Converged)
{
    throw new InvalidOperationException("Dominantes Eigenpaar ist nicht
    konvergiert.");
}

var deflation = WielandtDeflation.Create(
    matrix,
    dominant.Value,
    residualTolerance: 1e-8);

Console.WriteLine(deflation.SourceResidual);
```

Das Quellresiduum wird geprüft, weil aus einem beliebigen Vektor-/Eigenwertpaar zwar formal eine Matrix konstruiert werden könnte, daraus aber keine verlässliche Deflation des gewünschten Eigenwerts folgen würde.

Für den kompletten historischen Ablauf gibt es:

```
var second = EigenSolvers.WielandtSecondEigenpair(
    matrix,
    tolerance: 1e-11);
```

Dabei wird zunächst das dominante Paar berechnet, dann deflationiert, anschließend das dominante Paar der deflationierten Matrix gesucht und zuletzt der zweite Eigenvektor in die Koordinaten der Ausgangsmatrix zurückgeführt.

Bei mehrfachen oder nahezu mehrfachen dominanten Eigenwerten kann diese Rückabbildung schlecht konditioniert sein. Deshalb `second.Status` und `second.Message` auswerten und nicht jeden zurückgegebenen Wert automatisch als konvergiert betrachten.

7.8 *Vollständiges symmetrisches Eigensystem mit Jacobi*

Für eine reelle symmetrische Matrix garantiert der Spektralsatz reelle Eigenwerte und eine orthonormale Eigenvektorbasis. Das zyklische Jacobi-Verfahren nutzt diese Struktur und reduziert die Nebendiagonalelemente durch wiederholte Rotationen.

```

var decompositionResult = EigenSolvers.CyclicJacobi(
    matrix,
    tolerance: 1e-12,
    maximumSweeps: 100,
    symmetryTolerance: 1e-12);

if (!decompositionResult.Converged)
{
    Console.WriteLine(decompositionResult.Message);
}

var decomposition = decompositionResult.Value;
for (var i = 0; i < decomposition.Size; i++)
{
    var lambda = decomposition.GetEigenvalue(i);
    var vector = decomposition.GetEigenvector(i);
    var pair = decomposition.GetEigenpair(i);
    var equationResidual = EigenSolvers.EigenpairResidualNorm(matrix, pair);

    Console.WriteLine($"{i}: lambda={lambda}, Residuum={equationResidual}");
}

```

Wichtige Konventionen:

- Eigenwerte werden nach ihrem **numerischen Wert absteigend**, nicht nach dem Betrag sortiert;
- in `decomposition.Eigenvectors` stehen die Eigenvektoren spaltenweise;
- `GetEigenvalue` vermeidet eine komplette Eigenwert-Array-Kopie, wenn nur ein Skalar gebraucht wird;
- `GetEigenvector` liefert eine unabhängige Kopie;
- auch die öffentlichen Array-/Matrixeigenschaften der Zerlegung liefern defensive Kopien.

7.8.1 Was bedeutet das Jacobi-Residual?

Bei `CyclicJacobi` ist `decompositionResult.Residual` die Frobenius-Norm der Nebendiagonale der transformierten Arbeitsmatrix. Sie beschreibt, wie weit die Jacobi-Diagonalisierung fortgeschritten ist.

Dies ist eine andere Diagnose als $\|A \cdot v - \lambda \cdot v\|^2$. Zur unabhängigen Prüfung der finalen Eigenpaare sollte deshalb `EigenpairResidualNorm` wie im Beispiel aufgerufen werden.

7.9 Toleranzen sind Abbruchkriterien, keine garantierten Dezimalstellen

Eine Toleranz wie $1e-12$ ist ein internes Abbruchkriterium des numerischen Verfahrens. Sie verspricht nicht zwölf korrekte Dezimalstellen. Skalierung, Kondition, Rundungsfehler, spektrale Abstände und die Struktur der Matrix spielen ebenfalls eine Rolle.

Bei schwierigeren Problemen sind folgende Prüfungen sinnvoll:

1. Status, Iterations und das verfahrensspezifische Residuum ansehen;
2. zusätzlich `EigenpairResidualNorm` berechnen;

3. bei vollständigen symmetrischen Zerlegungen bei Bedarf die Orthogonalität der Eigenvektoren prüfen;
4. bei vermuteter Empfindlichkeit eine andere gültige Toleranz oder einen anderen Startvektor ausprobieren;
5. für folgenschwere oder große Rechnungen mit einer unabhängigen etablierten Bibliothek gegenprüfen.

7.10 Umfang und Performance

Die Verfahren sind für kleine und mittlere dichte Referenzprobleme, Lehre, deterministische SASD-Tests und spätere sprachübergreifende Konformitätsprüfungen gedacht. Sie verwenden absichtlich nachvollziehbare Schleifen statt Blockalgorithmen, SIMD-spezifischen Code oder native Abhängigkeiten.

Für große dichte, dünnbesetzte oder hochperformante Eigenwertprobleme sollte ein späteres SASD-Backend etablierte BLAS/LAPACK- bzw. spezialisierte Sparse-Bibliotheken verwenden und dabei klare SASD-Verträge beibehalten.

Gewöhnliche Differentialgleichungen und Randwertprobleme

Dieses Kapitel beschreibt die derzeit stabilen C#-APIs für Anfangswert- und Randwertprobleme. Es wächst weiter, sobald die noch fehlenden V1-Verfahren implementiert sind.

8.1 Anfangswertprobleme erster Ordnung

Ein skalares Anfangswertproblem erster Ordnung besteht aus

$$y' = f(x, y) \text{ und } y(x_0) = y_0.$$

Das Toolkit bietet derzeit das klassische Runge-Kutta-Verfahren vierter Ordnung mit fester Schrittweite (RK4), das adaptive Runge-Kutta-Fehlberg-Verfahren 4(5) (RKF45) sowie einen Adams-Bashforth-/Adams-Moulton-Prädiktor-Korrektor vierter Ordnung.

8.2 RK4 mit fester Schrittweite

```
using Sasd.Numerics.DifferentialEquations;

var points = RungeKutta.FourthOrder(
    (_, y) => y,
    x0: 0.0,
    y0: 1.0,
    xEnd: 1.0,
    step: 0.01);
```

Nur der letzte Schritt wird bei Bedarf verkürzt, damit die Folge genau bei xEnd endet.

8.3 Gleichungen zweiter Ordnung mit RK4

Ein skalares Anfangswertproblem zweiter Ordnung hat die Form $y'' = g(x, y, y')$ und benötigt $y(x_0)$ sowie $y'(x_0)$.

```
var points = RungeKutta.FourthOrderSecondOrder(
    (_, y, _) => -y,
    x0: 0.0,
    y0: 0.0,
    firstDerivative0: 1.0,
    xEnd: Math.PI / 2.0,
```

```
step: 0.01);
```

Intern wird die Gleichung zu $y'=v$, $v'=g(x,y,v)$ umgeformt und über denselben getesteten System-RK4-Kern integriert.

8.4 Gleichungen n-ter Ordnung mit RK4

Für $y^{(n)} = g(x, y, y', \dots, y^{(n-1)})$ erhält `FourthOrderNthOrder` die niedrigeren Ableitungen als geordneten Anfangszustand $[y, y', \dots, y^{(n-1)}]$.

```
var points = RungeKutta.FourthOrderNthOrder(
  (_, state) => -state[1],
  x0: 0.0,
  initialState: [0.0, 1.0, 0.0],
  xEnd: Math.PI / 2.0,
  step: 0.01);
```

Jeder `NthOrderOdePoint` speichert einen schreibgeschützten Snapshot des Zustands. Die eigentliche RK4-Logik bleibt im gemeinsamen Systemkern.

8.5 Gekoppelte Systeme zweiter Ordnung mit RK4

Mehrere gekoppelte Variablen zweiter Ordnung werden als $Y'' = G(x, Y, Y')$ formuliert.

```
var end = Math.PI / (2.0 * Math.Sqrt(2.0));
var points = RungeKutta.FourthOrderSecondOrderSystem(
  (_, values, _) =>
  {
    var difference = values[0] - values[1];
    return [-difference, difference];
  },
  x0: 0.0,
  initialValues: [1.0, -1.0],
  initialFirstDerivatives: [0.0, 0.0],
  xEnd: end,
  step: 0.01);
```

Die öffentlichen Werte- und Ableitungsvektoren bleiben getrennt, intern wird wieder auf ein System erster Ordnung reduziert.

8.6 Lineare Randwertprobleme mit Shooting

Das lineare Shooting behandelt

$$y'' = p(x)y' + q(x)y + r(x)$$

mit $y(x_0)=\alpha$ und $y(x_{\text{End}})=\beta$.

```
var result = LinearShooting.Solve(
  _ => 0.0,
  _ => 0.0,
```

```

_ => 2.0,
x0: 0.0,
leftValue: 1.0,
xEnd: 1.0,
rightValue: 4.0,
step: 0.05);

Console.WriteLine(result.InitialSlope);
Console.WriteLine(result.RightBoundaryResidual);

```

Das Verfahren integriert eine partikuläre und eine homogene Sensitivitätslösung. Wegen der Linearität kann die fehlende Anfangssteigung direkt aus beiden Lösungen kombiniert werden. `AuxiliaryRightValue` macht dabei eine singuläre oder schlecht konditionierte Randabbildung sichtbar.

8.7 Nichtlineare Randwertprobleme mit Shooting

Für

$$y'' = g(x, y, y')$$

mit $y(x_0)=\alpha$ und $y(x_{\text{End}})=\beta$ hängt der rechte Randwert im Allgemeinen nichtlinear von der unbekannten Anfangssteigung ab. `NonlinearShooting.Solve` verwendet deshalb zwei Startsteigungen und sucht mit dem Sekantenverfahren eine Nullstelle des Randresiduums

$$R(s) = y(x_{\text{End}}; s) - \beta.$$

```

// Exakte Lösung y=1/(1-x): y''=2y^3,
// y(0)=1, y(0.5)=2 und y'(0)=1.
var result = NonlinearShooting.Solve(
    (_, y, _) => 2.0 * y * y * y,
    x0: 0.0,
    leftValue: 1.0,
    xEnd: 0.5,
    rightValue: 2.0,
    step: 0.005,
    firstSlopeGuess: 0.5,
    secondSlopeGuess: 1.5,
    options: new NonlinearShootingOptions(
        BoundaryTolerance: 1e-10,
        MaximumIterations: 25));

if (result.Converged)
{
    Console.WriteLine(result.InitialSlope);           // ungefähr 1
    Console.WriteLine(result.RightBoundaryResidual); // nahe null
}
else
{
    Console.WriteLine($"{result.Status}: {result.Message}");
}

```

Nichtkonvergenz ist ein erwartbares numerisches Ergebnis und wird über `Status` gemeldet. `MaximumIterationsReached` bedeutet, dass die erlaubten Sekantenkorrekturen ausgeschöpft wurden. `NumericalBreakdown` kann auftreten, wenn zwei Randresiduen für

eine stabile Sekantenkorrektur zu ähnlich werden. Die letzte noch nutzbare Trajektorie bleibt im Ergebnis erhalten.

Ein Status Converged wird nur akzeptiert, wenn das tatsächliche Randresiduum die angeforderte BoundaryTolerance erfüllt. Die beiden Startsteigungen sind fachlich relevant: Bei mehreren Lösungen können unterschiedliche Startwerte zu unterschiedlichen Lösungen des Randwertproblems führen.

8.8 *Adaptives RKF45*

RKF45 ist sinnvoll, wenn automatische lokale Fehlerkontrolle und variable Schrittweiten wichtiger sind als eine feste Schrittweite.

```
var result = RungeKuttaFehlberg.Integrate(  
  (_, y) => y,  
  x0: 0.0,  
  y0: 1.0,  
  xEnd: 1.0,  
  new RungeKuttaFehlbergOptions  
  {  
    InitialStep = 0.25,  
    MinimumStep = 1e-8,  
    MaximumStep = 0.5,  
    AbsoluteTolerance = 1e-10,  
    RelativeTolerance = 1e-10  
  }  
);
```

Verworfenen Versuchsschritte sind bei einem adaptiven Verfahren normal. MinimumStepSizeReached bedeutet dagegen, dass die gewünschte lokale Toleranz mit der erlaubten Minimalweite nicht eingehalten werden konnte.

8.9 *Adams-Prädiktor-Korrektor*

Das Adams-Verfahren verwendet einen Adams-Bashforth-Prädiktor vierter Ordnung und einen Adams-Moulton-Korrektor vierter Ordnung. Die ersten drei Intervalle werden mit RK4 erzeugt, weil das Mehrschrittverfahren zunächst eine Historie benötigt.

```
var points = AdamsBashforthMoulton.Integrate(  
  (_, y) => y,  
  x0: 0.0,  
  y0: 1.0,  
  xEnd: 1.0,  
  maximumStep: 0.1);
```

Die AB4/AM4-Koeffizienten verlangen gleichmäßige Abstände. Deshalb wird das gesamte Gitter angepasst, wenn maximumStep das Intervall nicht exakt teilt.

8.10 *Wahl zwischen den Verfahren*

RK4 eignet sich als einfache feste Referenzrechnung. RKF45 bietet automatische lokale Fehlerkontrolle. Adams passt gut zu einem regelmäßigen Gitter und zur Wieder-

verwendung früherer Ableitungen. Lineares Shooting ist für lineare Gleichungen mit Wertbedingungen an beiden Enden gedacht; nichtlineares Shooting ergänzt diesen Fall, wenn die fehlende Anfangssteigung iterativ bestimmt werden muss.

Keines dieser Verfahren ist automatisch die richtige Wahl für steife Differentialgleichungen.

8.11 Randresiduum und globale Genauigkeit

Bei beiden Shooting-Verfahren bedeutet ein sehr kleines `RightBoundaryResidual` nur, dass die finale diskrete Trajektorie den verlangten Endwert gut trifft. Es ist keine Fehlerschranke für die inneren Punkte. Für belastbare Ergebnisse sollte mit kleinerer RK4-Schrittweite wiederholt und die fachlich wichtige Ergebnisgröße verglichen werden.

Beim nichtlinearen Shooting steuert `BoundaryTolerance` die Steigungssuche, während `step` den RK4-Diskretisierungsfehler beeinflusst. Nur eine dieser Größen zu verschärfen verbessert nicht automatisch die andere Fehlerquelle.

8.12 Derzeitige Grenzen

RK4 deckt skalare Gleichungen erster, zweiter und n-ter Ordnung sowie gekoppelte Systeme erster und zweiter Ordnung ab. RKF45 ist derzeit skalar und vorwärtsgerichtet, Adams skalar mit festem Gitter. Die historischen V1-Verfahren für lineares und nichtlineares Shooting bei skalaren Dirichlet-Randwertproblemen zweiter Ordnung sind nun beide implementiert. Allgemeinere Neumann-/Robin-Bedingungen, Multiple Shooting, Continuation-Verfahren und spezielle steife BVP-Solver liegen außerhalb des aktuellen V1-Kompatibilitätsziels.

Least-Squares-Approximation

Least-Squares-Verfahren passen ein Modell an Beobachtungen an, die sich normalerweise nicht alle exakt treffen lassen. Das SASD Math Toolkit unterstützt Polynomapproximationen, beliebige lineare Basisfunktionen und die benannten Kurvenmodelle aus dem klassischen Kompatibilitätsfundament.

9.1 Polynomapproximation

Für ein Polynom $y = c_0 + c_1x + \dots + c_nx^n$ wird `FitPolynomial` mit dem gewünschten Grad verwendet.

```
using Sasd.Numerics.Approximation;

double[] x = [-2, -1, 0, 1, 2];
double[] y = [17, 6, 1, 2, 9];

var coefficients = LeastSquares.FitPolynomial(x, y, degree: 2);
var predicted = LeastSquares.EvaluatePolynomial(coefficients, 1.5);
```

Die Koeffizienten werden nach steigender Potenz zurückgegeben. Die Polynom-Komfort-API verlangt weiterhin mindestens `degree + 1` Messwerte; sie wählt bei einem unterbestimmten Polynom nicht stillschweigend irgendeinen Koeffizientenvektor aus.

9.2 Potenz-, Exponential- und logarithmische Modelle

`FitPowerLaw` passt $y = a \cdot x^b$ über eine logarithmische Transformation. `FitExponential` passt $y = a \cdot \exp(b \cdot x)$ ebenfalls über den logarithmischen y-Raum. `FitLogarithmic` passt $y = a + b \cdot \ln(x)$ und transformiert nur x .

Die jeweiligen Domänenregeln bleiben wichtig: Potenzmodelle benötigen positive x - und y -Werte, Exponentialmodelle positive y -Werte und logarithmische Modelle positive x -Werte.

9.3 Fünfgliedrige Fourier-Anpassung

Für periodische Daten mit bekannter Grundperiode oder Kreisfrequenz steht

$$a_0 + a_1 \cos(w \cdot x) + b_1 \sin(w \cdot x) + a_2 \cos(2 \cdot w \cdot x) + b_2 \sin(2 \cdot w \cdot x)$$

zur Verfügung. Die Frequenz wird vom Aufrufer vorgegeben. Dieses Verfahren ist keine FFT und kann auch ungleichmäßig verteilte Messpunkte verwenden.

Der benannte Fourier-Helfer verlangt eine Designmatrix mit vollem Spaltenrang, weil

alle fünf benannten Koeffizienten einzeln identifizierbar sein sollen. Degenerierte Phasenwahlen werden deshalb weiterhin zurückgewiesen, obwohl die darunterliegende SVD mathematisch eine Minimalnormlösung aus unendlich vielen äquivalenten Koeffizientenvektoren auswählen könnte.

9.4 Beliebige lineare Basisfunktionen

Lässt sich ein Modell als $c_0 \cdot f_0(x) + c_1 \cdot f_1(x) + \dots$ formulieren, kann es direkt mit `FitBasis` angepasst werden. Jede Basisfunktion wird genau einmal pro Messpunkt ausgewertet und in der Designmatrix zwischengespeichert.

Die allgemeine Basis-API kann jetzt sowohl den gewöhnlichen Vollrangfall als auch schwierigere Rangstrukturen behandeln:

- quadratische/hohe Designmatrix mit vollem Spaltenrang -> Householder-QR;
- rangdefiziente oder unterbestimmte Designmatrix -> SVD-Minimalnormlösung.

Damit eignet sich `FitBasis` als allgemeiner Regressionsbaustein auch dann, wenn der Koeffizientenvektor nicht eindeutig ist.

9.5 Warum QR der bevorzugte Normalfall bleibt

Die historische Kompatibilitätsimplementierung verwendete Normalgleichungen:

$$(A^T A)c = A^T y.$$

Durch die explizite Bildung von $A^T A$ wird jedoch die Konditionszahl quadriert. Ein nur mäßig schwieriges Regressionsproblem kann dadurch numerisch deutlich problematischer werden.

Für quadratische bzw. hohe Designmatrizen mit vollem Spaltenrang faktorisiert die modernisierte Implementierung die Designmatrix direkt als $A = Q \cdot R$ mit Householder-Reflexionen. Nach Anwendung von Q^T auf die Beobachtungen bleibt ein oberes Dreieckssystem. QR ist günstiger als eine vollständige SVD und bleibt daher der bevorzugte Standard, solange der Rang unproblematisch ist.

`QrFactorization` steht zusätzlich öffentlich unter `Sasd.Numerics.LinearAlgebra` zur Verfügung, wenn eine Anwendung die Zerlegung selbst benötigt oder für mehrere rechte Seiten wiederverwenden möchte.

9.6 Wann die SVD übernimmt

Meldet QR eine Rangdefizienz oder ist die Designmatrix breit bzw. unterbestimmt, verwendet die allgemeine Basis-Engine `SingularValueDecomposition`.

Die SVD liefert Singulärwerte und einen expliziten toleranzabhängigen numerischen Rang. Komponenten an oder unterhalb der eingestellten Rangschwelle werden abgeschnitten. Der zurückgegebene Koeffizientenvektor ist dann die Minimum-Euklidische-Norm-Lösung für die beibehaltenen Richtungen.

Das ist eine wichtige Unterscheidung: Bei einem rangdefizienten Modell ist der Koeffizientenvektor nicht eindeutig, selbst wenn die Vorhersagen vollständig bestimmt

sind. Die SVD macht die Auswahl explizit, statt einen willkürlichen Koeffizientensatz als eindeutig erscheinen zu lassen.

Direkter Zugriff ist ebenfalls möglich:

```
var svd = SingularValueDecomposition.Decompose(design);  
Console.WriteLine(svd.EstimatedRank);  
Console.WriteLine(svd.ConditionNumber);  
  
var coefficients = svd.SolveLeastSquares(observations);
```

Weitere Details zu Pseudoinverser und Konditionszahl stehen im Kapitel zur modernen dichten linearen Algebra.

9.7 Residuen und Transformationen verstehen

Potenzgesetz- und Exponentialanpassung transformieren y und minimieren deshalb Fehler im logarithmischen y -Raum. Ihre ausgegebenen RSS-/RMSE-Werte werden anschließend im ursprünglichen y -Raum berechnet.

Logarithmische und fünfgliedrige Fourier-Anpassung verändern y nicht. Bei ihnen entspricht `ResidualSumOfSquares` direkt der gewöhnlichen Least-Squares-Zielfunktion in den ursprünglichen y -Einheiten.

9.8 Praktische Prüfung

Ein Fit sollte nicht nur anhand seiner Parameter beurteilt werden. Residuen sollten geprüft oder geplottet werden. Auch mit QR und SVD bleibt eine sinnvolle Skalierung wichtig, und ein numerisch erfolgreicher Fit beweist nicht, dass das gewählte Modell fachlich geeignet ist.

Bei rangempfindlichen Aufgaben sollten Singulärwerte und Rang betrachtet werden, statt einen zurückgegebenen Koeffizientenvektor als Beweis dafür zu verstehen, dass jeder Parameter unabhängig messbar ist. Der historische Modellumfang bleibt erhalten; die weitere Entwicklung konzentriert sich jetzt auf Regressionsdiagnostik, Gewichtung, robuste Verfahren und die breiteren Statistik-Anforderungen der SASD Statistical Workbench.

FFT, Faltung und Korrelation

Die Fast Fourier Transform (FFT) überführt eine abgetastete Folge aus dem Zeit-/Samplebereich in eine diskrete Frequenzdarstellung. Das SASD Math Toolkit verwendet derzeit eine gut lesbare Radix-2-Cooley-Tukey-Referenzimplementierung.

10.1 Komplexe FFT

Für komplexe Samples werden Forward und Inverse verwendet:

```
using System.Numerics;
using Sasd.Numerics.Transforms;

Complex[] samples = [1, 2, 3, 4];
var spectrum = FastFourierTransform.Forward(samples);
var restored = FastFourierTransform.Inverse(spectrum);
```

Die Vorwärtstransformation ist unskaliert. Die inverse Transformation teilt durch die Folgenlänge, sodass ein Vorwärts-/Rückwärtsdurchlauf die ursprüngliche Folge bis auf Gleitkomma-Rundungsfehler rekonstruiert.

Jede nicht leere Folge muss derzeit eine Zweierpotenz als Länge besitzen.

10.2 Reelle Eingabe: vollständiges oder kompaktes Spektrum

ForwardReal ist die direkte Kompatibilitäts-API. Sie wandelt die reellen Samples in komplexe Werte um und liefert alle N Fourier-Bins.

Für die meisten Anwendungen ist ForwardRealCompact günstiger:

```
using Sasd.Numerics.Transforms;

double[] samples = [1.0, 0.0, -1.0, 0.0, 1.0, 0.0, -1.0, 0.0];
var spectrum = FastFourierTransform.ForwardRealCompact(samples);

Console.WriteLine(spectrum.OriginalLength); // 8
Console.WriteLine(spectrum.BinCount);      // 5

for (var k = 0; k < spectrum.BinCount; k++)
{
    Console.WriteLine($"bin={k}, f/Fs={spectrum.GetNormalizedFrequency(k)},
        magnitude={spectrum[k].Magnitude}");
}
```

Bei reeller Eingabe sind die negativen Frequenzbins die komplex konjugierten Gegenstücke der positiven Frequenzbins. Beide Hälften zu speichern wäre redundant. `RealFftSpectrum` speichert deshalb nur Gleichanteil bis Nyquist.

Bei einer Abtastrate `sampleRate` ergibt sich die physikalische Frequenz des Bins `k` mit

```
var frequencyHz = spectrum.GetNormalizedFrequency(k) * sampleRate;
```

10.3 Reelles Signal rekonstruieren

Die kompakte Darstellung wird mit `InverseReal` zurücktransformiert:

```
var restored = FastFourierTransform.InverseReal(spectrum);
```

Das Toolkit rekonstruiert intern die ausgelassene konjugierte Hälfte. Anwendungscode muss das Packformat nicht kennen.

10.4 FFT im Unterschied zur Fourier-Least-Squares-Approximation

Dieses Kapitel darf nicht mit `LeastSquares.FitFiveTermFourier` verwechselt werden. Die FFT analysiert Frequenzbins einer regelmäßig abgetasteten Folge. Das fünfgliedrige Least-Squares-Modell passt dagegen bei einer vorgegebenen Grundfrequenz ein kleines periodisches Modell an und kann auch unregelmäßige x-Koordinaten verwenden. Trotz Sinus-/Kosinusbezug lösen beide Verfahren unterschiedliche Aufgaben.

10.5 Lineare Faltung

Die Faltung kombiniert zwei endliche Folgen, beispielsweise ein Signal und eine Impulsantwort. Reelle und komplexe Überladungen liefern jeweils die vollständige lineare Faltung:

```
using System.Numerics;
using Sasd.Numerics.Transforms;

double[] signal = [1.0, 2.0, 1.0];
double[] kernel = [0.5, -0.5];
var filtered = FastFourierTransform.ConvolveReal(signal, kernel);

Complex[] complexSignal = [new(1, 1), new(2, -1)];
Complex[] complexKernel = [new(3, 0), new(0, -1)];
var complexFiltered = FastFourierTransform.ConvolveComplex(complexSignal,
    complexKernel);
```

Bei nicht leeren Eingaben enthält das Ergebnis `left.Count + right.Count - 1` Samples. Die Eingaben müssen nicht selbst auf eine Zweierpotenz aufgefüllt werden; der Helfer wählt intern eine passende FFT-Länge und entfernt die Auffüllung anschließend wieder.

10.6 Kreuzkorrelation und Lag-Reihenfolge

Die Kreuzkorrelation misst Ähnlichkeit, während eine Folge relativ zur anderen verschoben wird. Dafür stehen `CrossCorrelateReal` und `CrossCorrelateComplex` bereit.

Bei komplexen Samples verwendet das Toolkit die Konjugation

```
r[l] = sum_k left[k] * conjugate(right[k-l]).
```

Das Ergebnis ist von Lag $-(\text{right.Count} - 1)$ bis $\text{left.Count} - 1$ geordnet. Für einen Ergebnisindex i ergibt sich der Lag mit

```
var lag = i - (right.Count - 1);
```

Der Wert bei Lag null befindet sich damit am Index $\text{right.Count} - 1$. Bei komplexen Daten entspricht er dem überlappenden komplexen inneren Produkt.

```
var correlation = FastFourierTransform.CrossCorrelateComplex(complexSignal,
    complexKernel);
var zeroLag = correlation[complexKernel.Length - 1];
```

Korrelationsbibliotheken verwenden unterschiedliche Konventionen. Anwendungscode, der Peak-Positionen interpretiert, sollte diese Index-Lag-Zuordnung daher ausdrücklich berücksichtigen.

10.7 Leere und ungültige Eingaben

Faltung oder Korrelation mit einer leeren Folge liefert ein leeres Ergebnis. Reelle und komplexe Transformationshelfer weisen NaN und Unendlich zurück. Bei direkten FFT-Aufrufen muss die Länge einer nicht leeren Eingabe weiterhin eine Zweierpotenz sein.

10.8 Praktische Grenzen

Die aktuelle Implementierung priorisiert Verständlichkeit und Testbarkeit statt maximalen FFT-Durchsatz. Sie erzeugt Zwischenarrays; auch die kompakte Real-FFT berechnet derzeit zunächst eine vollständige Transformation und verwirft danach die redundante Hälfte, und die Faltung verwendet auch bei sehr kleinen Folgen immer die FFT. Spätere optimierte oder native Backends können diese Details verbessern, ohne die öffentliche API zu ändern.

Bei der Interpretation eines Spektrums sollten Abtastrate, Fensterfunktion, Skalierung und spektrale Leckage ausdrücklich berücksichtigt werden. Die FFT transformiert die gelieferten Samples; sie wendet nicht automatisch eine Fensterfunktion an und leitet auch kein physikalisches Abtastintervall her.

Numerische Rezepte, Diagnostik und typische Fehlerfälle

Eine numerische Routine liefert selten nur eine Zahl. Eine belastbare Anwendung muss zusätzlich wissen, **wie das Verfahren beendet wurde, wie gut der zurückgegebene Wert das mathematische Problem erfüllt, welche Toleranz verwendet wurde und ob das Problem selbst möglicherweise schlecht konditioniert ist.**

Dieses Kapitel führt die Diagnosekonventionen des SASD Math Toolkit an einer Stelle zusammen. Es ersetzt die Fachkapitel nicht, sondern liefert eine gemeinsame Lesart für Ergebnisse aus Nullstellensuche, Integration, linearer Algebra, Eigenwertverfahren, ODE-Solvern und Approximation.

11.1 1. Vier Fragen vor der Übernahme eines numerischen Ergebnisses

Bevor ein Rückgabewert als brauchbar gilt, sollten vier getrennte Fragen beantwortet werden:

1. **War der Aufruf gültig?** Falsche Dimensionen, nicht-endliche Eingaben, unmögliche Optionen oder ein ungültiges Intervall sind Vertragsverletzungen und führen normalerweise zu einer .NET-Exception.
2. **Wie wurde das Verfahren beendet?** Iterative und adaptive Routinen liefern Statuswerte wie `Converged`, `MaximumIterationsReached`, `NumericalBreakdown`, `NotBracketed`, `MaximumDepthReached` oder `MinimumStepSizeReached`.
3. **Wie groß ist der Defekt?** Ein Residuum oder eine Fehlerschätzung sagt etwas über die Näherung aus, aber seine Bedeutung ist problemspezifisch.
4. **Ist das Problem für die gewünschte Interpretation ausreichend gut konditioniert?** Ein kleines Residuum bedeutet nicht automatisch einen kleinen Fehler in der gesuchten Größe.

Diese Fragen sind in der API absichtlich getrennt. Ein Status ist kein Residuum, ein Residuum ist kein Vorwärtsfehler, und eine Toleranz ist keine Garantie für eine bestimmte Zahl korrekter Dezimalstellen.

11.2 2. Exceptions und Ergebnisstatus haben unterschiedliche Aufgaben

Für die C#/.NET-Implementierung gilt folgende Grundregel:

- **Ungültige API-Aufrufe oder ungültige numerische Eingaben** werden mit normalen .NET-Exceptions wie `ArgumentException` oder `ArgumentOutOfRangeException`

abgelehnt.

- **Erwartbare numerische Ergebnisse nach einem gültigen Aufruf** werden normalerweise über Ergebnisstatus gemeldet.
- **Direkte Arithmetik ohne sinnvolles endliches Ergebnis** kann `ArithmeticException` auslösen, wenn kein nützlicher iterativer Status zurückgegeben werden kann.

Ein Bisection-Intervall ohne Vorzeichenwechsel ist beispielsweise ein mathematisch sinnvoller Ausgang. `RootResult.Status` meldet deshalb `IterationStatus.NotBracketed`. Eine Eingabe `double.NaN` ist dagegen ein ungültiger Aufruf und wird bereits vor Beginn der Iteration abgelehnt.

Für Anwendungscode ist diese Trennung wichtig. Exceptions sollten nicht der normale Kontrollfluss für „nicht konvergiert“ sein; umgekehrt sollten Exceptions nicht so behandelt werden, als seien sie lediglich ein weiterer Konvergenzstatus.

11.3 3. Converged ist eine Beendigungsinformation, kein Exaktheitsbeweis

Das generische `IterativeResult<T>` enthält Status, Iterations, Value, ein optionales Residual und eine optionale Diagnose Message.

```
var result = LinearSystemSolvers.GaussSeidel(  
    matrix,  
    rightHandSide,  
    tolerance: 1e-10,  
    maximumIterations: 500);  
  
if (!result.Converged)  
{  
    Console.WriteLine($"Abbruch mit {result.Status}: {result.Message}");  
}  
  
if (result.HasFiniteResidual)  
{  
    Console.WriteLine($"Residuum = {result.Residual:E3}");  
}
```

Converged bedeutet, dass das Verfahren **sein dokumentiertes Konvergenzkriterium** erfüllt hat. Das Ergebnis ist deshalb nicht mathematisch exakt. Konditionierung, Gleitkommarundung, Modellfehler, Diskretisierungsfehler und ungeeignete Skalierung können weiterhin relevant sein.

Umgekehrt kann ein nicht konvergiertes Ergebnis durchaus eine brauchbare letzte Näherung und ein endliches Residuum enthalten. Die neue Eigenschaft `HasFiniteResidual` ist deshalb bewusst unabhängig von `Converged`.

11.4 4. Residuum, Vorwärtsfehler und Fehlerschätzung sind nicht dasselbe

11.4.1 Nullstellensuche

Für eine skalare Nullstellennäherung x ist `RootResult.Residual`

$|f(x)|$.

Ein kleiner Wert zeigt, dass x die Gleichung $f(x)=0$ gut erfüllt. Er misst jedoch nicht direkt $|x-x^*|$ zur exakten Nullstelle x^* . In der Nähe mehrfacher Nullstellen oder bei sehr kleiner Ableitung kann ein kleines Funktionsresiduum mit einem deutlich größeren Fehler in x zusammenfallen.

```
var root = RootSolvers.NewtonRaphson(
    x => Math.Cos(x) - x,
    x => -Math.Sin(x) - 1.0,
    0.5);
```

```
Console.WriteLine(root.Status);
Console.WriteLine(root.Root);
Console.WriteLine(root.Residual);
```

11.4.2 Lineare Gleichungssysteme

Für $A \cdot x = b$ berechnet `LinearSystemSolvers.ResidualInfinityNorm`

$||A \cdot x - b||_{\infty}$.

Das ist ein Gleichungsdefekt und keine direkte Schranke für $||x - x^*||$. Bei einer schlecht konditionierten Matrix kann ein kleines Residuum trotzdem zu einer empfindlichen Lösung gehören.

```
var x = LinearSystemSolvers.SolveGaussian(matrix, rightHandSide);
var residual = LinearSystemSolvers.ResidualInfinityNorm(matrix, x, rightHandSide);
```

Bei direkten Solvern ist diese nachträgliche Residuenberechnung ein sinnvoller unabhängiger Anwendungstest, weil der direkte Solver selbst kein iteratives Ergebnisobjekt zurückgibt.

11.4.3 Eigenpaare

Für $A \cdot v = \lambda \cdot v$ berechnet `EigenSolvers.EigenpairResidualNorm`

$||A \cdot v - \lambda \cdot v||_2$.

```
var result = EigenSolvers.PowerMethod(matrix);
var residual = EigenSolvers.EigenpairResidualNorm(matrix, result.Value);
```

Ein kleines Eigenpaarresiduum bestätigt die Eigenwertgleichung in absolutem Sinn. Es sagt jedoch nicht, dass der Eigenwert unempfindlich gegenüber Störungen ist. Mehrfache oder eng benachbarte Eigenwerte können insbesondere einzelne Eigenvektoren grundsätzlich empfindlich machen.

11.4.4 Adaptive Integration

`AdaptiveIntegrationResult.EstimatedError` und `RombergIntegrationResult.EstimatedError` sind **algorithmische Fehlerindikatoren**, die aus Verfeinerung bzw. Extrapolation entstehen. Sie sind kein Vergleich mit einem dem Programm unbekannten exakten Integral.

Wenn eine analytische Lösung oder eine deutlich genauere unabhängige Referenz vorhanden ist, sollte sie zusätzlich verwendet werden.

11.4.5 Adaptive ODE-Integration

Runge-Kutta-Fehlberg steuert eine **lokale Schrittfehlerschätzung**. Eine erfolgreiche lokale Fehlerkontrolle macht die gesamte berechnete Trajektorie nicht exakt. AdaptiveOdeResult meldet deshalb, ob der Endpunkt erreicht wurde und wie viele Versuchsschritte akzeptiert oder verworfen wurden. Für wichtige Anwendungen können zusätzlich Invarianten, analytische Lösungen oder unabhängige feinere Rechnungen geprüft werden.

11.5 5. Eine Toleranz muss zur Problemskala passen

Eine Zahl wie $1e-10$ hat allein keine universelle Bedeutung.

Bei einem Problem mit natürlichen Werten um $1e-6$ und einem anderen um $1e12$ beschreibt dieselbe absolute Schwelle völlig unterschiedliche Anforderungen. Einige Toolkit-Verfahren verwenden bewusst absolute Toleranzen, weil dies der klarste historische bzw. Referenzvertrag ist. Adaptives RKF bietet dagegen getrennte absolute und relative Toleranzen, weil dort beide benötigt werden.

Praktisch gilt:

- In der Methodendokumentation nachsehen, **welche Größe mit der Toleranz verglichen wird**.
- Toleranzen aus der physikalischen oder mathematischen Skala des Problems ableiten.
- NumericConstants.DefaultTolerance nicht als Zusage einer festen Anzahl richtiger Dezimalstellen interpretieren.
- NumericConstants.NearlyZero nicht als allgemeine Gleitkomma-Gleichheitsprüfung verwenden.

NearlyZero ist ein Schutzwert der Referenzimplementierung für numerisch vernachlässigbare Pivots, Divisoren und Normen. Öffentliche APIs wie LU-Faktorisierung und inverse Iteration besitzen eine explizite Pivot-Toleranz, wenn die Anwendung diese Entscheidung an ihre Problemskala anpassen muss.

11.6 6. Häufige Beendigungsstatus und sinnvolle Reaktionen

11.6.1 MaximumIterationsReached

Das Verfahren blieb numerisch sinnvoll, erfüllte sein Kriterium aber nicht innerhalb des Limits.

Sinnvolle Fragen sind: Ist die Toleranz realistisch? Ist der Startwert schlecht? Konvergiert das Verfahren grundsätzlich langsam? Passt das Verfahren zum Problem? Wird das Residuum noch kleiner?

Einfach nur das Iterationslimit zu erhöhen ist erst dann sinnvoll, wenn diese Fragen

geklärt sind.

11.6.2 NumericalBreakdown

Eine notwendige numerische Voraussetzung ist verloren gegangen: ein Divisor wurde vernachlässigbar klein, ein Iterat nicht-endlich, eine Zwischenrechnung lief über oder eine andere Invariante ging verloren.

Nicht automatisch mit sehr viel lockerer Toleranz neu starten. Zuerst Diagnosemeldung und Skalierung prüfen. Ein Breakdown weist häufig auf ein strukturelles Problem hin.

11.6.3 NotBracketed

Ein bracketing-basiertes Nullstellenverfahren erhielt ein Intervall ohne erforderlichen Vorzeichenwechsel. Ein besseres Intervall wählen oder mit Fachwissen nach einer Klammer suchen. Dieser Status bedeutet nicht „die Funktion besitzt nirgends eine Nullstelle“.

11.6.4 MaximumDepthReached / NumericalResolutionReached

Adaptive Quadratur konnte die normale Verfeinerung nicht fortsetzen. Der Rückgabewert ist die beste am Grenzpunkt verfügbare Näherung, aber Converged ist falsch. Ursachen können Singularitäten, Unstetigkeiten, starke Oszillation oder eine unrealistische Toleranz sein.

11.6.5 MinimumStepSizeReached

Die adaptive ODE-Integration müsste unter die konfigurierte minimale Schrittweite gehen, um die lokale Fehlertoleranz einzuhalten. Toleranz, Mindestschritt und Verhalten der Differentialgleichung prüfen. Das kann auch auf Steifheit hindeuten; RKF45 ist kein spezieller steifer Solver.

11.6.6 Singuläre oder numerisch singuläre Matrix

Direkte Verfahren können ArithmeticException auslösen, wenn ein Pivot unter die konfigurierte Schwelle fällt. Zu unterscheiden sind eine tatsächlich singuläre Matrix, eine stark schlecht konditionierte Matrix und eine Matrix, deren Skalierung nicht zur gewählten absoluten Pivot-Toleranz passt.

11.7 7. Nicht-endliche Werte sind Fehlerzustände, keine normalen Daten

Die numerischen V1-Verfahren erwarten endliche Gleitkommawerte, sofern eine API nichts anderes dokumentiert. NaN und Unendlich werden an öffentlichen Grenzen soweit sinnvoll abgelehnt. Liefert ein Callback der Anwendung während einer Rechnung einen nicht-endlichen Wert, meldet das Verfahren je nach Vertrag einen numerischen Breakdown oder eine Arithmetic-Exception.

Diese Politik ist beabsichtigt. Eine stillschweigende NaN-Fortpflanzung über viele Iterationen kann einen formal abgeschlossenen Ablauf erzeugen, ohne dass noch eine

brauchbare Diagnose möglich ist.

Auch Anwendungen sollten importierte Daten vor der Rechnung prüfen und genügend Kontext aufbewahren, um die ursprüngliche Zeile, Probe, Messung oder den Parameter zu identifizieren.

11.8 8. Konditionierung ist etwas anderes als algorithmische Konvergenz

Ein stabiles Verfahren kann ein schlecht konditioniertes Problem nicht gut konditioniert machen.

Beispiele:

- nahezu singuläre lineare Systeme können große Änderungen in x aus kleinen Änderungen in A oder b erzeugen;
- mehrfache oder eng benachbarte Polynomnullstellen reagieren empfindlich auf Koeffizientenänderungen;
- mehrfache oder geclusterte Eigenwerte können einzelne Eigenvektoren instabil machen;
- numerische Differentiation verstärkt Rauschen;
- hochgradige Polynominterpolation kann ein schlechtes Modell sein, obwohl die Interpolationsgleichungen exakt gelöst wurden.

„Der Solver ist konvergiert“ ist deshalb nur ein Teil der Qualitätsaussage.

11.9 9. Unabhängige Prüfrezepte

Für wichtige Rechnungen sollte die zweite Prüfung möglichst nicht nur dasselbe Abbruchkriterium wiederholen.

11.9.1 Nullstelle

Originalfunktion an der gemeldeten Nullstelle auswerten und `RootResult.Residual` protokollieren. Wenn eine Ableitung verfügbar ist, prüfen, ob die Funktion dort nahezu flach ist.

11.9.2 Direktes lineares Gleichungssystem

$\|A*x-b\|_\infty$ mit `ResidualInfinityNorm` neu berechnen. Bei sensiblen Anwendungen zusätzlich eine anders skalierte Formulierung oder eine unabhängige hochwertige Linear-Algebra-Bibliothek vergleichen.

11.9.3 Eigenpaar

`EigenpairResidualNorm` auf der Originalmatrix verwenden. Bei einer vollständigen symmetrischen Zerlegung zusätzlich Orthogonalität und gegebenenfalls Rekonstruktion von A prüfen.

11.9.4 Integration

Bei schwierigen Integranden mit strengerer Toleranz oder einem anderen Verfahren wiederholen. Die Übereinstimmung verschiedener Verfahren ist stärker als nur ein größeres Rekursionslimit desselben Verfahrens.

11.9.5 ODE

Bekannte Invarianten, analytische Lösung, feinere Schrittweite oder unabhängigen Solver vergleichen. Bei langen Integrationen den Fehlerverlauf untersuchen und nicht nur den letzten Punkt.

11.9.6 Least Squares

Residuen im **ursprünglichen Datenraum** betrachten, nicht nur in transformierten Koordinaten. Die benannten transformierten SASD-Modelle berichten ihre Diagnostik deshalb im ursprünglichen y-Raum.

11.10 10. Ein praktisches Abnahmemuster

```
var result = EigenSolvers.PowerMethod(
    matrix,
    tolerance: 1e-10,
    maximumIterations: 500);

if (!result.Converged)
{
    throw new InvalidOperationException(
        $"Eigenwertberechnung endete mit {result.Status}: {result.Message}");
}

var residual = EigenSolvers.EigenpairResidualNorm(matrix, result.Value);
if (residual > 1e-8)
{
    throw new InvalidOperationException(
        $"Eigenpaarresiduum {residual:E3} überschreitet die Anwendungsgrenze.");
}

var eigenpair = result.Value;
```

Wichtig ist die Trennung zwischen **Algorithmustoleranz** und **Abnahmegrenze der Anwendung**. Die Bibliothek steuert ihren numerischen Prozess; die Anwendung entscheidet, ob das Ergebnis für ihren fachlichen, wissenschaftlichen oder technischen Zweck gut genug ist.

11.11 11. Was protokolliert werden sollte

Für später reproduzierbare numerische Ergebnisse sollten mehr Daten als nur der Endwert gespeichert werden:

- Algorithmusname und relevante Optionen;

- Eingabeskala oder Datensatzidentität;
- Beendigungsstatus;
- Anzahl Iterationen, Verfeinerungen oder Schritte;
- Residuum bzw. Fehlerschätzung, wenn vorhanden;
- Diagnosemeldung;
- Toolkit-Version bzw. Commit bei Forschungs- oder Validierungsabläufen.

Nur `success=true` zu speichern verwirft genau die Informationen, die bei späteren Abweichungen am hilfreichsten sind.

11.12 12. Kurze Prüfliste

Vor der Übernahme eines Ergebnisses prüfen:

- Eingaben sind endlich und erfüllen den API-Vertrag;
- der Beendigungsstatus ist verstanden;
- Residuum/Fehlerindikator hat die erwartete Definition und Skalierung;
- die Toleranz passt zum Problem;
- Iterations-, Tiefen- oder Schrittgrenzen haben das Verfahren nicht verfrüht beendet;
- keine Diagnosemeldung weist auf einen Breakdown hin;
- Konditionierung oder Datenrauschen entwerfen die Interpretation nicht;
- bei wichtigen Ergebnissen existiert eine unabhängige Prüfung.

Diese Diagnosehaltung ist bewusst Teil der Architektur des SASD Math Toolkit. V1 bevorzugt explizites, prüfbares numerisches Verhalten gegenüber APIs, die nur eine nackte Zahl zurückgeben und verbergen, wie sie entstanden ist.

Moderne dichte lineare Algebra – Erweiterungen 2026

Das klassische Matrixkapitel bleibt gültig. Die Modernisierung ergänzt Zerlegungen, die besser zu heutigen Workloads passen, und koppelt die Verfahrenswahl stärker an die Struktur der Matrix statt jedes Problem als allgemeine Gauß-Elimination zu behandeln.

12.1 Householder-QR

Allgemeines und polynomiales Least Squares bevorzugen inzwischen Householder-QR statt Normalgleichungen. QR vermeidet die Bildung von $A^T A$, quadriert die Konditionszahl also nicht nur für die Anpassung, und stellt eine praktische Vollrangdiagnostik bereit.

```
var qr = QrFactorization.Decompose(design);
if (qr.IsFullColumnRank)
{
    var coefficients = qr.SolveLeastSquares(observations);
}
```

Für gewöhnliche quadratische/hohe Vollrang-Regression bleibt QR der bevorzugte Ausgangspunkt, weil es günstiger als eine vollständige SVD ist.

12.2 Cholesky

Für eine symmetrisch positiv definite Matrix:

```
var factorization = CholeskyFactorization.Decompose(a);
var x = factorization.Solve(b);
```

Die Faktorisierung speichert $A = L L^T$, kann mehrere rechte Seiten ohne erneute Zerlegung lösen und bietet `LogDeterminant()` für skalierungsempfindliche Statistikaufgaben.

Cholesky sollte nicht auf eine lediglich symmetrische oder beliebige Matrix erzwungen werden. Positive Definitheit gehört zum mathematischen Vertrag und ist kein Implementierungsdetail.

12.3 Singulärwertzerlegung

Die SVD übernimmt Fälle, in denen Rang und Kondition selbst Teil des Problems sind:

```
var svd = SingularValueDecomposition.Decompose(a);

Console.WriteLine(svd.EstimatedRank);
Console.WriteLine(svd.ConditionNumber);

var minimumNorm = svd.SolveLeastSquares(b);
var pseudoInverse = svd.PseudoInverse();
```

Für eine $m \times n$ -Matrix werden dünne Faktoren bereitgestellt mit

$$A = U * S * V^T$$

und $\min(m,n)$ absteigend sortierten Singulärwerten.

Die Managed-Referenzimplementierung verwendet eine einseitige Jacobi-SVD. Die Zerlegung wird bewusst nicht über $A^T A$ gewonnen; die Normalmatrix würde die Konditionszahl quadrieren und kann gerade schwache Singulärrichtungen verlieren, die mit einer SVD diagnostiziert werden sollen.

12.3.1 Numerischer Rang ist toleranzabhängig

EstimatedRank basiert auf

```
singularValue > LargestSingularValue * RelativeRankTolerance
```

Der resultierende absolute Grenzwert steht als RankThreshold zur Verfügung. Das ist eine numerische Entscheidung. Ein mathematisch von null verschiedener Singulärwert kann bewusst verworfen werden, wenn er relativ zur dominanten Skala zu klein für eine stabile Lösung ist.

Dieselbe Schwelle steuert SolveLeastSquares und PseudoInverse, damit Diagnose und Berechnung dieselben Singulärrichtungen verwenden.

12.3.2 Rangdefizientes Least Squares

Für rangdefiziente oder unterbestimmte Probleme liefert SolveLeastSquares die Minimum-Euklidische-Norm-Lösung zu den beibehaltenen Singulärrichtungen. Das ist oft sinnvoller, als willkürlich eine Untermenge von Spalten auszuwählen.

Beispiel:

```
var a = new DenseMatrix(new double[,]
{
    { 1.0, 0.0, 1.0 },
    { 0.0, 1.0, 1.0 }
});

var svd = SingularValueDecomposition.Decompose(a);
var x = svd.SolveLeastSquares(new[] { 1.0, 1.0 });
```

Das System besitzt unendlich viele exakte Lösungen. Die SVD liefert die Minimalnormlösung ungefähr (1/3, 1/3, 2/3).

12.3.3 Pseudoinverse

`PseudoInverse()` bildet die Moore-Penrose-Pseudoinverse mit derselben Abschneideschwelle. Sie ist sinnvoll, wenn eine nachfolgende Methode tatsächlich die Matrix benötigt. Für wiederholte rechte Seiten sollte die SVD wiederverwendet und `SolveLeastSquares` aufgerufen werden, statt jedes Mal die komplette Pseudoinverse zu materialisieren.

12.3.4 Konditionszahl

Für eine numerisch vollrangige Matrix liefert `ConditionNumber` das Verhältnis vom größten zum kleinsten Singulärwert und damit die 2-Norm-Konditionszahl. Bei numerischer Rangdefizienz wird positive Unendlichkeit gemeldet; `ReciprocalConditionNumber` ist dann null.

Eine große Konditionszahl bedeutet, dass kleine Störungen der Eingangsdaten deutlich größere Änderungen der Lösung verursachen können. Sie bewertet nicht die Qualität der Implementierung, sondern die Empfindlichkeit des mathematischen Problems.

12.4 Matrixnormen und gemeinsamer Diagnosebericht

Moderne numerische Anwendungen müssen häufig mehr beantworten als „hat der Solver einen Vektor geliefert?“. Matrixskalierung, Rang und Nullraumdimension entscheiden oft darüber, ob ein Ergebnis fachlich belastbar ist.

Der Helfer `MatrixNorms` bietet die gebräuchlichsten Normen:

```
var one = MatrixNorms.OneNorm(a);
var infinity = MatrixNorms.InfinityNorm(a);
var frobenius = MatrixNorms.FrobeniusNorm(a);
var spectral = MatrixNorms.SpectralNorm(a);
```

`OneNorm` ist die größte absolute Spaltensumme, `InfinityNorm` die größte absolute Zeilensumme, `FrobeniusNorm` die euklidische Norm aller Einträge und `SpectralNorm` der größte Singulärwert. Frobenius- und Absolutsummenberechnung verwenden skalierte Akkumulation, damit große endliche Einträge nicht unnötig früh zu Zwischenüberläufen führen.

Wer mehrere Diagnosen gemeinsam benötigt, sollte eine einzige Analyse verwenden:

```
var diagnostics = MatrixConditionDiagnostics.Analyze(a);

Console.WriteLine(diagnostics.EstimatedRank);
Console.WriteLine(diagnostics.LeftNullity);
Console.WriteLine(diagnostics.RightNullity);
Console.WriteLine(diagnostics.ConditionNumber2);
```

Der gemeinsame Bericht berechnet die günstigen eintragsbasierten Normen direkt und verwendet genau eine SVD gemeinsam für Spektralnrm, Rang und 2-Norm-Kondition. Dadurch werden versehentlich mehrfach ausgeführte identische Zerlegungen vermieden.

Linke und rechte Nullität werden bewusst getrennt dargestellt. Eine breite 2x3-Matrix kann vollen rechteckigen Rang 2 und trotzdem rechte Nullität 1 besitzen. Bei einem

unterbestimmten System beschreibt dieser rechte Nullraum genau die Richtungen, die zu einer Lösung addiert werden können, ohne die rechte Seite zu verändern.

Ein kleines Residuum und eine große Konditionszahl können gleichzeitig auftreten. Das Residuum sagt, dass der berechnete Vektor die dargestellten Gleichungen gut erfüllt; die Konditionszahl sagt, dass die dargestellten Gleichungen selbst empfindlich auf kleinste Störungen reagieren können.

12.5 *Least-Squares-Routing im Toolkit*

Die allgemeine API `LeastSquares.FitBasis` wählt den dichten Solver jetzt bewusst:

1. quadratische/hohe Designmatrix mit vollem Spaltenrang: Householder-QR;
2. rangdefiziente oder unterbestimmte Designmatrix: SVD-Minimalnormlösung.

Benannte Modelle dürfen strengere Identifizierbarkeitsregeln behalten. So weist der fünfgliedrige Fourier-Helfer weiterhin eine Phasenwahl zurück, die nicht alle fünf benannten Koeffizienten bestimmen kann, statt still eine von unendlich vielen äquivalenten Koeffizientenlösungen zurückzugeben.

12.6 *Verfahrensauswahl*

Problemstruktur	Sinnvoller Ausgangspunkt
Allgemeines quadratisches dichtes System	Pivotierte LU
Symmetrisch positiv definites System	Cholesky
Quadratisches/hohes Vollrang-Least-Squares	Householder-QR
Rangdefizientes Least Squares	SVD
Unterbestimmtes Minimalnorm-System	SVD
Matrixskalierung / 1-, Unendlich-, Frobeniusnorm	MatrixNorms
Numerischer Rang / 2-Norm-Kondition	SVD oder MatrixConditionDiagnostics
Explizite Pseudoinverse benötigt	SVD
Sehr großer dichter Produktions-Workload	Später optionaler BLAS/LAPACK-artiger Back

Die Verfahren ergänzen sich. Die SVD liefert die reichhaltigste Diagnose der drei modernen Zerlegungen, ist aber nicht die billigste Lösung für jedes gut strukturierte System.

12.7 *Performance-Perspektive*

Die aktuellen QR-, Cholesky- und SVD-Implementierungen sind abhängigkeitsfreie Managed-Referenzalgorithmen. Sie vermeiden offensichtliche unnötige Arbeit und numerisch ungünstige Formulierungen, sollen aber bei sehr großen Matrizen keine hochoptimierte Hersteller-BLAS/LAPACK schlagen. Die SASD-Strategie bleibt: nachvollziehbare deterministische Referenzschicht behalten, reale Workloads benchmarken und später optionale beschleunigte Backends hinter stabilen Konzepten ergänzen.

Dünnbesetzte lineare Algebra – CSR, CG/PCG, GMRES und BiCGSTAB

Dichte Matrizen reservieren Speicher für jede Matrixposition. Das ist für viele kleine und mittlere Probleme sinnvoll, wird aber verschwenderisch, wenn eine große Matrix pro Zeile nur wenige von null verschiedene Koeffizienten enthält. `CsrMatrix` ist die erste SASD-Sparse-Repräsentation für solche Workloads.

13.1 Sparse-Matrix aufbauen

Für einen koordinatenorientierten Aufbau:

```
using Sasd.Numerics.LinearAlgebra.Sparse;

var a = CsrMatrix.FromEntries(
    4,
    4,
    [
        new SparseMatrixEntry(0, 0, 4.0),
        new SparseMatrixEntry(0, 1, -1.0),
        new SparseMatrixEntry(1, 0, -1.0),
        new SparseMatrixEntry(1, 1, 4.0),
        new SparseMatrixEntry(1, 2, -1.0),
        new SparseMatrixEntry(2, 1, -1.0),
        new SparseMatrixEntry(2, 2, 4.0),
        new SparseMatrixEntry(2, 3, -1.0),
        new SparseMatrixEntry(3, 2, -1.0),
        new SparseMatrixEntry(3, 3, 4.0)
    ]
);
```

Nach dem Aufbau ist die Matrix unveränderlich. Die Einträge liegen in kanonischem CSR vor: sortierte Spalten je Zeile, keine doppelten Koordinaten und keine explizit gespeicherten Nullwerte.

Doppelte Koordinaten werden in `FromEntries` addiert. Das ist nützlich, wenn mehrere lokale Beiträge denselben globalen Matrixkoeffizienten verändern. Heben sie sich exakt auf, verschwindet der Eintrag aus der komprimierten Speicherung.

13.2 Matrix-Vektor-Multiplikation

Der bequeme Standardaufruf erzeugt einen neuen Ergebnisvektor:

```
var y = a.Multiply(x);
```

Iterative Algorithmen sollten Arbeitsbereiche dagegen wiederverwenden:

```
var y = new double[a.Rows];  
a.Multiply(x, y);
```

Diese zweite Form wird von iterativen Sparse-Solvern verwendet, damit wiederholte Matrix-Vektor-Produkte nicht in jeder Iteration neue Ergebnisarrays anlegen.

13.3 Dense-Konvertierung und Sparsifizierung

`CsrMatrix.FromDense(dense)` speichert jeden von null verschiedenen Dense-Eintrag. Eine positive `absoluteZeroTolerance` verwirft bewusst kleinere Koeffizienten:

```
var sparse = CsrMatrix.FromDense(dense, absoluteZeroTolerance: 1e-10);
```

Diese Schwelle ist absolut. Ihre Wahl gehört zum numerischen Modell; ein einziges Epsilon ist nicht automatisch für Matrizen mit völlig unterschiedlichen physikalischen Skalen geeignet.

`ToDense()` eignet sich für Inspektion, Interoperabilität und Tests. Sparse-Algorithmen sollten aber nicht fortlaufend in dichte Speicherung zurückkonvertieren.

13.4 Transponieren

`Transpose()` liefert wieder kanonisches CSR für die transponierte Matrix, ohne zunächst eine dichte Matrix anzulegen. Eine eigene CSC-Struktur kann später folgen, wenn dauerhaft spaltenorientierte Workloads sie rechtfertigen.

13.5 Sparse-Normen

`SparseMatrixNorms` berechnet preiswerte Normen direkt aus der komprimierten Speicherung:

```
var one = SparseMatrixNorms.OneNorm(a);  
var infinity = SparseMatrixNorms.InfinityNorm(a);  
var frobenius = SparseMatrixNorms.FrobeniusNorm(a);
```

Die Frobenius-Berechnung ist skalierungsbewusst. Große, aber endliche Einträge laufen deshalb nicht nur deshalb über, weil eine naive Zwischenrechnung sie zuerst quadriert.

13.6 Conjugate Gradient für SPD-Systeme

`ConjugateGradientSolver` löst Sparse-Systeme, deren Matrix **symmetrisch positiv definit** ist. Er ist kein allgemeiner Ersatz für LU oder einen nichtsymmetrischen Krylov-Solver.

```
var result = ConjugateGradientSolver.Solve(  
    a,
```

```
rightHandSide,
options: new SparseIterativeSolverOptions
{
    AbsoluteTolerance = 1e-12,
    RelativeTolerance = 1e-10,
    MaximumIterations = 1000
});
```

Die gemeinsame Sparse-Abbruchregel lautet:

$$||b - A*x||_2 \leq \max(\text{AbsoluteTolerance}, \text{RelativeTolerance} * ||b||_2)$$

Symmetrie wird standardmäßig geprüft. Zusätzlich wird eine strikt positive Diagonale verlangt, weil sie für positive Definitheit notwendig ist. Diese preiswerten Prüfungen können nicht beweisen, dass eine beliebige Sparse-Matrix SPD ist; deshalb meldet CG zusätzlich `NumericalBreakdown`, wenn die Suchrichtungskrümmung $p^T A p$ nicht positiv oder nicht endlich wird.

13.6.1 Verifikation des echten Residuums

CG aktualisiert sein Residuum aus Effizienzgründen rekursiv. Floating-Point-Rundung kann dazu führen, dass dieses rekursive Residuum vom echten $b - A*x$ abweicht. Vor `Converged` berechnet die SASD-Implementierung deshalb das echte Residuum neu. Liegt es weiterhin oberhalb der Schwelle, startet CG mit dem aktualisierten Residuum neu, statt falsche Konvergenz zu melden.

13.7 Preconditioned Conjugate Gradient

Schlechte Skalierung oder ein ungünstiges Spektrum können dazu führen, dass CG trotz einer SPD-Matrix viele Iterationen benötigt. Ein Preconditioner wendet eine angenäherte Inversoperation an:

$$z = M^{-1} * r$$

Die öffentliche Schnittstelle `ISparsePreconditioner` ist solverneutral. Die erste Implementierung ist `JacobiPreconditioner`, der die inverse Matrixdiagonale verwendet:

```
var jacobi = JacobiPreconditioner.Create(a);
var result = ConjugateGradientSolver.SolvePreconditioned(
    a,
    rightHandSide,
    jacobi,
    options: new SparseIterativeSolverOptions
    {
        AbsoluteTolerance = 1e-12,
        RelativeTolerance = 1e-10,
        MaximumIterations = 1000
    });
```

Jacobi ist billig aufzubauen und anzuwenden, erzeugt pro Anwendung keine neuen Arrays und eignet sich besonders gut als Skalierungs-Baseline. Er garantiert nicht bei jedem Problem weniger Iterationen; die Qualität eines Preconditioners ist problemabhängig.

Für PCG muss der Preconditioner selbst SPD sein. Führt ein benutzerdefinierter Preconditioner dazu, dass $r^T M^{-1} r$ nicht positiv oder nicht endlich ist, meldet der Solver `NumericalBreakdown`.

13.8 Restarted GMRES für allgemeine quadratische Systeme

`GmresSolver` ist das allgemeine nichtsymmetrische Gegenstück zu CG/PCG. Der Solver setzt weder Symmetrie noch positive Definitheit voraus und minimiert das Residuum über einen Krylov-Unterraum, der mit dem Arnoldi-Prozess aufgebaut wird.

```
var result = GmresSolver.Solve(
    matrix,
    rightHandSide,
    restartLength: 30,
    options: new SparseIterativeSolverOptions
    {
        AbsoluteTolerance = 1e-12,
        RelativeTolerance = 1e-10,
        MaximumIterations = 1000
    });
```

Vollständiges GMRES speichert pro Iteration einen zusätzlichen Basisvektor. `restartLength` begrenzt dieses Speicherwachstum. An einer Restart-Grenze wird die aktuelle Least-Squares-Korrektur angewendet, das echte Residuum neu berechnet und der nächste Krylov-Zyklus von diesem Residuum aus gestartet.

Die Arnoldi-Basis verwendet zwei Modified-Gram-Schmidt-Pässe, um vermeidbaren Orthogonalitätsverlust zu reduzieren. Givens-Rotationen aktualisieren das kleine obere Hessenberg-Least-Squares-System inkrementell. Das projizierte Residuum dient als günstiger Auslöser, aber `Converged` wird erst nach expliziter Neuberechnung des echten Residuums $b - A \cdot x$ gemeldet.

13.8.1 Right-preconditioned GMRES

Dieselbe solverneutrale Preconditioner-Abstraktion wird über `SolvePreconditioned` wiederverwendet. GMRES wendet den Preconditioner auf der rechten Seite an:

$$A * M^{-1} * y = b$$

Dadurch bleibt das minimierte und gemeldete Residuum das Residuum des ursprünglichen Systems. Anders als PCG verlangt GMRES keinen SPD-Preconditioner.

```
var jacobi = JacobiPreconditioner.Create(matrix);
var result = GmresSolver.SolvePreconditioned(
    matrix,
    rightHandSide,
    jacobi,
    restartLength: 30);
```

Ein abgebrochener Arnoldi-Ausbau wird nicht automatisch als Erfolg gewertet. Kann der Krylov-Raum nicht weiter wachsen und liegt das explizit neu berechnete Residuum noch über der Toleranz, liefert der Solver `NumericalBreakdown` mit der besten

verfügbaren Lösung.

13.9 BiCGSTAB für nichtsymmetrische Systeme mit begrenztem Speicher

BiCgStabSolver ist ein weiterer allgemeiner nichtsymmetrischer Krylov-Solver. Anders als GMRES verwendet er eine kurze Rekurrenz und hält deshalb nur eine feste Anzahl von $O(n)$ -Arbeitsvektoren. Das kann bei großen Systemen interessant sein, wenn der Speicherbedarf einer GMRES-Basis relevant wird.

```
var result = BiCgStabSolver.Solve(
    matrix,
    rightHandSide,
    options: new SparseIterativeSolverOptions
    {
        AbsoluteTolerance = 1e-12,
        RelativeTolerance = 1e-10,
        MaximumIterations = 1000
    });
```

BiCGSTAB minimiert kein wachsendes Least-Squares-Problem direkt. Der Residualverlauf kann deshalb unruhiger als bei GMRES sein, und die kurze Rekurrenz besitzt mehrere skalare Breakdown-Punkte. Null oder nichtendliche Werte beim Shadow-Residual-Produkt, Alpha-Nenner oder Stabilisationskoeffizienten werden als NumericalBreakdown gemeldet und nicht mit einem Iterationslimit verwechselt.

13.9.1 Right-preconditioned BiCGSTAB

BiCGSTAB verwendet ebenfalls ISparsePreconditioner und wendet M^{-1} auf seine Suchrichtungen an, bevor mit A multipliziert wird:

```
var jacobi = JacobiPreconditioner.Create(matrix);
var result = BiCgStabSolver.SolvePreconditioned(
    matrix,
    rightHandSide,
    jacobi);
```

Wie bei GMRES muss der Preconditioner nicht SPD sein. Konvergenz wird trotzdem über das echte Residuum der ursprünglichen Gleichung gemeldet. Behauptet nur das rekursive Residuum Konvergenz, die explizite Neuberechnung aber nicht, startet BiCGSTAB vom aktualisierten echten Residuum neu.

13.9.2 Auswahl des Sparse-Solvers

Eine praktische erste Regel ist:

- **CG/PCG** für SPD-Systeme;
- **restarted GMRES** für allgemeine nichtsymmetrische Systeme, wenn robuste Residualminimierung Priorität hat;
- **BiCGSTAB** für allgemeine nichtsymmetrische Systeme, wenn begrenzter Basis-Speicher besonders wichtig ist und sein Konvergenzverhalten für das Problem passt.

Nicht allein nach der Iterationszahl auswählen. Speicher, Matvec-Kosten, Preconditioner-Qualität und das gemeldete echte Residuum gehören gemeinsam zur Beurteilung.

13.9.3 Solver-Diagnostik

`SparseLinearSolveResult` wird von CG, PCG, GMRES und BiCGSTAB gemeinsam verwendet. Es enthält die beste verfügbare Lösung, `IterationStatus`, Anzahl abgeschlossener Iterationen, Anfangs-/Endresiduum, Norm der rechten Seite, effektive Konvergenzschwelle und relative Residualnorm. `MaximumIterationsReached` bleibt damit quantitativ auswertbar und wird nicht auf ein Boolean reduziert.

13.10 Was die Sparse-Schicht noch nicht liefert

CSR, CG/PCG, die gemeinsame Preconditioner-Abstraktion, Jacobi-Preconditioning, restarted GMRES und BiCGSTAB sind jetzt implementiert. Als Nächstes folgt eine Konsolidierungs- und Referenzfallrunde vor dem Release Candidate. Leistungsfähigere Preconditioner wie Incomplete Cholesky/ILU sowie eine eigene CSC-Struktur bleiben aufgeschoben, bis konkrete Workloads sie rechtfertigen.

Die Trennung bleibt wichtig: Sparse-Speicherung ist eine Datenlayout-Entscheidung, die Solverwahl hängt von der mathematischen Struktur ab, und die Wahl des Preconditioners hängt sowohl vom Solver-Vertrag als auch von der Matrix ab.

Historischer Kontext und aktuelle V1-Abdeckung

Die C#-Ausgabe folgt auf hoher Ebene bewusst den großen numerischen Problemklassen der historischen Numerical Methods Toolbox. Die Umsetzung ist jedoch keine Quellcode-Portierung. Öffentliche APIs, Datentypen, Fehlerbehandlung, Beispiele und Diagnosen gehören zum SASD Math Toolkit. Die Modernisierung 2026 geht dort über den historischen Katalog hinaus, wo heutige SASD-Anwendungen stärkere numerische Grundlagen benötigen.

Themenblock	C#/.NET-Ausgabe
Nullstellen	Bisektion, Newton-Raphson, Sekante, Newton-Horner, Muller, Laguerre und Polynomdeflation
Interpolation	Lagrange, dividierte Differenzen, natürliche und geklemmte kubische Splines
Differentiation / Integration	Funktions-, Tabellen- und Spline-Differentiation; Newton-Cotes, adaptive Regeln, Romberg und Gauss-Legendre
Dichte Matrixverfahren	DenseMatrix, Gauß, pivotierte LU, Householder-QR, Cholesky, SVD, Pseudoinverse, Normen, Kondition, Inverse, Residuen und Gauss-Seidel
Sparse-Matrixverfahren	Kanonisches CSR, Sparse-Matvec/Normen, Jacobi-Preconditioning, CG/PCG, restarted GMRES, BiCG-STAB und unabhängige True-Residual-Diagnostik
Eigenwertprobleme	Potenz-/inverse Potenzmethode, Wielandt-Deflation und zyklisches Jacobi-Verfahren
Differentialgleichungen	RK4-Familie, RKF45, Adams-Prädiktor-Korrektor sowie lineares und nichtlineares Shooting
Least Squares	Polynom- und Basisfits sowie benannte Modelle; Householder-QR bei vollem Spaltenrang mit SVD-Fallback für rangdefiziente oder unterbestimmte Minimalnormprobleme
FFT / Anwendungen	Komplexe und reelle FFT, kompaktes Realspektrum, Faltung und Kreuzkorrelation
Demonstrationen	Plattformunabhängiger deterministischer HTML/SVG-Bericht plus moderner Sparse-Release-Smoke-Solve

Build und Pflege der LaTeX-Ausgabe

Die LaTeX-Ausgabe wird absichtlich aus den bestehenden Markdown-Kapiteln generiert. Änderungen an fachlichen Erläuterungen sollen daher zuerst in docs/user-guide/de/ bzw. docs/user-guide/en/ erfolgen. Das Verzeichnis LaTeX/ enthält Layout, Buildlogik und sprachspezifische Front-/Back-Matter.

```
cd docs/user-guide/LaTeX
make de      # deutsches PDF
make en      # English PDF
make all     # beide Ausgaben
```


Lizenz und Herkunft

Das SASD Math Toolkit steht unter der MIT-Lizenz. Historisches Borland-Material und historische Produktnamen bleiben Eigentum ihrer jeweiligen Rechteinhaber. Dieses Handbuch dokumentiert die unabhängig entwickelte SASD-C#/.NET-Implementierung und reproduziert nicht den historischen Handbuchtext.